YenkoExpress

UK shops, paid in cedis, delivered in Ghana

Complete Documentation

YenkoExpress: overview, guides, operations, architecture, security and the Software Requirements Specification

Contents	13 documents in one book
Audience	Owner and staff, the developer, and anyone reviewing the system
Generated	12 October 2026

Describes the system as built. This is not legal, tax or customs advice. Generated from the Markdown files in the repository's docs folder, so it is only as current as the date above; regenerate it with `npm run docs:pdf` after documents change.

Contents

1. Overview

- What the system is
- The money: how a customer's total is worked out
- The life of an order
- Where products come from
- Who uses it
- Glossary

2. Admin guide

- Contents
- 1. Signing in
- 2. Your first day: set-up checklist
- 3. The daily routine
- 4. Every page, one by one
- 5. Link orders in detail
- 6. Using the admin on a phone
- 7. Staff accounts and access
- 8. Good habits

3. Customer guide

- What this shop does
- Finding something
- Can't find it here? Paste the link
- Ordering
- Paying
- Tracking your order
- Cancelling and refunds
- Your account
- Your information
- Reviews
- Staying safe
- Getting help

4. Catalogue sources

- The idea
- Choosing the right kind
- Setting up a source
- What runs by itself
- Rules the importer follows (and will not break)
- Removing a source
- Troubleshooting a source

5. Deployment and operations

- 5.1 What the server needs
- 5.2 The current deployment (Render)

5.3 Settings reference
5.4 First deployment, step by step
5.5 Scheduled jobs
5.6 Backups (set up before you take orders)
5.7 Monitoring
5.8 Runbooks
5.9 Costs to expect
5.10 Not covered yet

6. Architecture

Technology
Folder map
The main modules in src/lib
How a request flows
An order, end to end
The catalogue importer (src/lib/ingest)
Background work
Access control in the code
Design decisions worth knowing

7. Data model

How the tables relate
What each table is for
Columns

8. Security and privacy

8.1 Who can sign in, and how
8.2 Access control (roles and rights)
8.3 Money and payments
8.4 Secrets
8.5 Other protections
8.6 Personal data held
8.7 Not covered (be aware)
8.8 If something goes wrong

9. Integrations

How the page works
Payments
Messages: SMS, WhatsApp, email
Exchange rate feed (optional)
eBay (catalogue API)
Diffbot (Product API, paid)
Customer sign-in: Google, Facebook and Apple
Webhooks and scheduled jobs: addresses
Adding a new provider (for developers)

10. Testing

Commands
Layer 1: unit and database tests

Layer 2: end-to-end browser run
What is deliberately not tested automatically
Manual checklist before announcing the shop
Adding tests

11. FAQ and troubleshooting

For the operator
For customers (quick answers)
Error messages you may see

12. Change history

Foundations
Deployment fixes
Products and photos
Catalogue sources
Shops and delivery
Link orders
Ordering from Amazon UK
Staff and access
Phones
Branding
Documentation and testing
Known gaps (carried forward)

Software Requirements Specification

Contents
1. Introduction
2. Overall description
3. External interface requirements
4. Functional requirements
5. Verification and traceability
6. Non-functional requirements
7. Data requirements
8. Business rules
9. Use cases
10. Constraints, assumptions, dependencies and risks
Appendix A: Requirement status summary
Appendix B: Gaps and future work
Appendix C: Glossary
Appendix D: Revision history

1. Overview

What the system is

A website where a shopper in Ghana can:

- browse shops and products that you list, with every price shown in **pounds and cedis**;
- send a link to **any** UK product, even from a shop you do not list, and get a price;
- check out and pay once, in cedis;
- track the order from the UK shop to their door.

You (the operator) buy the items from the UK retailers, ship them to Ghana, and a third-party courier makes the last delivery. The site does not hold stock.

The money: how a customer's total is worked out

```
customer total = items + service charge + shipping to Ghana + delivery in Ghana
```

Part	How it is worked out	Where you set it
Items	UK price in pounds × quantity, converted to cedis at the **effective rate** (your rate plus your markup).	Admin → Pricing (exchange rate and markup)
Service charge	Your fee for buying on the customer's behalf: a percentage (with a minimum), a flat amount, or tiered bands by item total. Always shown as its own line.	Admin → Pricing
Shipping to Ghana	From the chosen shipping method's **rate card**, by chargeable weight (the larger of actual and volumetric weight; today items carry their actual weight, so that is what is used). A parcel up to a bracket's weight costs that bracket's price; above the largest bracket there is a charge per extra kilogram; there is a minimum charge.	Admin → Shipping
Delivery in Ghana	A flat fee for the chosen delivery area.	Admin → Delivery areas

All money is stored as whole units (pence for pounds, pesewas for cedis), so there are no rounding drifts. The same calculation runs on the product page, the cart, checkout and when the order is saved, so the figure a customer sees is the figure they are charged. The order stores the rate, markup and every amount at the moment it was placed, so later changes to your settings never alter an old order.

Starting values for a new database (change them on first day): service charge 10% with a GH₵30 minimum; exchange rate GH₵15.20 per £1 with a 3% markup; minimum order £10 of items; two shipping methods (Air freight, 8 to 12 days; Express air, 4 to 7 days) with sample rate cards; five delivery areas from GH₵30 to GH₵120.

Customs duty and taxes charged in Ghana are **not** included in the total. The shop says so at checkout.

The life of an order

```
Customer pays ⟶ Payment received ⟶ Buying from the UK shop ⟶ Bought from the UK shop
      ⟶ Received at our UK address ⟶ On its way to Ghana ⟶ Clearing customs in Ghana
      ⟶ Out for delivery ⟶ Delivered
```

Rules the system enforces:

- **Only a confirmed payment** (from the payment gateway) moves an order to *Payment received*. Staff cannot mark an order paid by hand.
- Staff move an order **one step forward at a time**. They can **cancel** it up to *Buying from the UK shop*. A cancelled order that was paid can then be marked **Refunded** after you refund the customer through your gateway.
- Staff can add **tracking entries** at four stages: *UK shop dispatch*, *Received at our UK address*, *UK to Ghana shipment* and *Delivery in Ghana* (carrier, reference and a link).
- Each status change can send the customer an SMS, WhatsApp message and/or email, depending on your rules and the customer's choices.
- Customers see the same stages in their account, with the notes you add.

Where products come from

You decide which UK shops to show and fill them with products from **catalogue sources**: eBay's official API, Diffbot's Product API, Shopify and WooCommerce shops, product feeds, file imports and pasted links. The importer keeps prices and stock fresh and follows strict rules (it obeys `robots.txt` and stops when a shop refuses). See Catalogue sources.

Shops that **block** automated reading (many large retailers do) are served by **link orders**: the customer pastes the product link, the system or your team prices it, and the customer pays through normal checkout. See Admin guide → Link requests.

Who uses it

Person	What they do
Customer	Browses, pastes links, orders, pays, tracks, reviews delivered items. Needs an account to check out.
Super admin	The developer. Signs in with `ADMIN_PASSWORD`. Can do everything and is the only one who can create staff accounts.
Staff	Signed in with email and password. Each person only has the rights the super admin gave them (for example *Customer support* or *Finance*).

Glossary

Term	Meaning
Catalogue source	One place the importer is allowed to read a shop's products from.
Link order	An order for an item the customer found elsewhere and sent as a link, priced by a quote.
Quote	The UK price and weight of one item, entered by your team or filled in automatically, plus a private pay link for the customer.

Term	Meaning
Effective rate	The exchange rate customers actually pay: your rate with your markup added.
Rate card	The shipping price list for one method, by weight.
Chargeable weight	The larger of the real weight and the volumetric weight, used for shipping.
Super admin	The developer's sign-in; not stored in the database.
Right (permission)	One thing a staff account may do, such as "Update orders".
Gateway	A payment provider: Stripe, Paystack or Flutterwave.
Outbox	The queue of messages waiting to be sent to customers, with retries.
Held item	An imported item waiting in Import review because its price looked odd or jumped.

2. Admin guide

For the people who run the shop. Everything is under **/admin**. What you can see depends on your account: the menu only shows pages you are allowed to open.

Contents

1. Signing in
2. Your first day: set-up checklist
3. The daily routine
4. Every page, one by one
5. Link orders in detail
6. Using the admin on a phone
7. Staff accounts and access
8. Good habits

---

1. Signing in

Who	Where	With
Staff	`/admin/login`	Their email and password.
Super admin (the developer)	`/admin/login?developer=1` (the "Developer sign-in" link under the form)	The `ADMIN_PASSWORD` set on the server.

- A new staff member signs in with the first password the super admin gave them and is **asked to choose their own** before anything else opens (minimum 10 characters).
- After **5 wrong attempts** from one place, or 8 for one email, sign-in is blocked for 15 minutes.
- Sessions last 8 hours. **Sign out** is in the sidebar (on a phone, under **Menu**).
- Change your own password any time under **My account**.

2. Your first day: set-up checklist

Do these in order before announcing the shop.

1. **Pricing** (Admin → Pricing): set the exchange rate, your markup, the service charge and the minimum order. Set the site name.
2. **Shipping** (Admin → Shipping): replace the sample rate cards with your real prices per weight bracket.
3. **Delivery areas** (Admin → Delivery areas): your real areas and delivery fees.
4. **Integrations** (Admin → Integrations): add **at least one payment gateway**, then message providers (SMS, WhatsApp, email). Use each card's **Test** button. Without a gateway customers cannot pay.
5. **Shops and catalogue sources**: create shops, then add sources so they fill with products (Catalogue sources).
6. **Appearance**: pick a colour theme.
7. **Staff accounts** (super admin): create an account for each person with only the access they need.

8. **Place one real small order** with a real payment, and follow it through every status, before telling customers.

3. The daily routine

Open the **Dashboard** first. The **Needs attention** list tells you what to do, most urgent first:

Item	What to do
Paid orders that need a refund	Orders cancelled after payment. Refund the customer in your gateway's dashboard, then mark the order **Refunded**.
Payments with the wrong amount	A gateway reported a different amount than the site asked for. Check before shipping anything.
Paid orders waiting to be bought	Buy these from the UK shops, then move each order on.
New link requests	Check the price and stock, then send a quote (unless it was quoted automatically).
Orders with no movement for 5+ days	Chase the shop, forwarder or courier.
Imported items waiting for review	Approve or reject them in **Import review**.
Catalogue sources that stopped	A shop refused access or a feed failed. Open the source to see why.
Messages that could not be sent	Check the provider keys, then retry.
Unpaid orders older than 24 hours	A nudge may help.
Exchange rate gap	Shown when your rate differs from the market by the alert percentage.

Typical order, start to finish

1. A paid order appears as **Payment received**. Open it, buy the items from the UK shop, and click **Mark as: Buying from the UK shop**, then **Bought from the UK shop**. Put the UK order reference in the note (the customer sees notes).
2. When the parcel reaches your UK address, add a **tracking entry** (*Received at our UK address*) and move the status on.
3. When it ships, add the *UK to Ghana shipment* tracking and mark **On its way to Ghana**; then **Clearing customs**, **Out for delivery** (with the courier's reference under *Delivery in Ghana*), and **Delivered**.
4. Open **What it cost us** on the order and enter the real costs, so the margin is right.

4. Every page, one by one

Dashboard

Key figures for the last 7, 30 or 90 days (revenue, orders, average order, service charge earned, new customers, each compared with the period before), a sales chart, **Orders in progress** by status, **Where the money comes from** (items, service charge, shipping, delivery), **Margin** (from the costs you enter), top shops and items, delivery areas, the exchange rate, customers, integrations status, latest orders and recent admin activity. **Export orders (CSV)** appears if you have that right.

Orders

Search by order number, name or phone and filter by status. Open an order to see items, delivery details, the price breakdown, the payment attempts, and the full event history.

- **Status buttons** show only the legal next steps. A note is optional; **the customer sees it**.
- **Tracking**: add or remove entries for the four stages (carrier, reference, link, note).
- **What it cost us**: shop price paid (£), UK delivery (£), the rate you bought at, freight, Ghana delivery, payment fees and other costs. The page then shows the **margin**. Needs the *Record costs and see margins* right.
- A **yellow banner** marks a **link order** and says how the price was found (see section 5).

Customers

Search and open a customer: their orders, saved addresses and details, and two tools: **Help with sign-in** (create a one-time password-reset link to send them yourself) and **Account status** (disable or enable the account; disabling signs the customer out everywhere and does not affect their orders).

Link requests

Everything customers send by link. See section 5.

Shops

Add, edit and **delete** shops: name, category (this is the **department** shown on the site), tagline, description, website, colour, order on the page, a **logo** (upload a JPEG, PNG, WebP or GIF up to 4 MB, or paste a link) and whether the shop is shown. Deleting a shop removes its items, its sources and their cart/wishlist entries; **orders already placed keep their own record**. You must tick a confirmation. If you only want to hide a shop, untick "Show this shop to customers".

Items

All products. Add an item by hand (name, brand, category, description, UK price, weight, size/colour options, source link, photo upload or link, shown or hidden) or edit existing ones. **Add by link** reads a pasted product link. Imported items appear here too; changes you make by hand to imported items can be overwritten by the next source refresh.

Catalogue sources

Where products come from. Create, check ("Check this setup first" shows what would be read without saving), run now, switch on or off, upload files for file-import sources, and **remove** a source (keeping or removing the products it brought in). Full details in Catalogue sources.

Import review

Items waiting for a person: odd prices (under 50p, over £10,000), names that look like a warranty, protection plan or fee rather than a product, price jumps over the source's limit (default 40%), updates you chose to approve by hand. **Approve** puts the item or its new price live; **Reject** keeps it off. There is a bulk approve.

Reviews

Customer reviews, filterable by **All / Published / Hidden**. Only customers whose order was **delivered** can review that item, once. Hide a review to remove it from the product page.

Pricing

- **Service charge**: percentage with a minimum, a flat amount, or tiered bands ("up to £50 pay 15%…"). Only the chosen method is used.
- **Exchange rate and minimum order**: GH₵ per £1, the markup, and the smallest order (items only, in £).
- **Site**: site name, the support WhatsApp number, a **contact email** and the **registered business name**. The email and business name appear in the public privacy policy and terms, so fill them in before you submit the policy links to Facebook, Google or Apple.
- **What customers pay now**: a live example so you can see the effect of a change before saving.
- The **exchange-rate feed** policy (manual, suggest or automatic, with a maximum automatic move and an alert gap) is under Integrations.

Shipping

Shipping methods and their **rate cards**: weight brackets with a price each, a charge per extra kilogram above the biggest bracket, and a minimum charge. Add a method, edit one, or hide one.

Delivery areas

The areas customers choose at checkout, each with a name, the places included, the Ghana delivery fee, the usual delivery time, order and whether it is offered. You can add, edit and **delete** areas. You cannot delete the **last active** area (checkout needs somewhere to deliver). Orders already placed keep the area name and fee they were charged.

Appearance

Choose one of **15 colour themes**. It applies to the whole shop at once.

Integrations

Keys for every outside service, stored **encrypted** and shown only masked. An environment setting on the server always wins over a saved one. Sections: **Readiness** (what is missing), **Payment gateways**, **Messaging channels**, **Catalogue APIs** (eBay), **Exchange rate feed**, and **Which updates are sent** (per order status, which channels are used). Each card shows the webhook address to register and a **Test** button. Details in Integrations.

Messages

The log of every SMS, WhatsApp message and email sent to customers (recipients are partly hidden), with status. Failed messages are tried three times, then wait for you: **Retry** one or **Send queued messages now**.

Activity log

Who did what and when: sign-ins, settings changes, order status changes, staff changes, exports. Search it. Staff actions show the person's name and email; automatic actions show "system".

Staff accounts (super admin only)

See section 7.

My account

Your name, role and rights, and **Change password**.

5. Link orders in detail

A **link order** is an order for an item that is **not in your catalogue**. It exists because many big retailers block automated reading, so you cannot list their products, but you can still buy from them for customers.

How the customer starts

They paste a product link into the big box on the home page, the header's **Add by link** button, the search bar, or the **Paste link** tab on a phone (or open `/request`). If the system can read the page, they see the name and the price in pounds and cedis and can continue in one step.

What the system does

For each request the system saves the link, the quantity, the size/colour notes, the **kind of item**, and who sent it. Then:

- If it **can read the price from the shop's own page**, and that is allowed by your rules, it **quotes by itself** and the customer can pay immediately.
- If it cannot, and **customer-typed prices** are allowed, it quotes from the price the customer typed **plus your safety margin**.
- Otherwise the request waits in **Link requests** for your team.

Your team's part (Admin → Link requests)

1. Open the link on the shop and check the real price and stock.
2. Enter the **UK price of one item** and its **weight** (grams). Choose how many **days** to hold the price and add an optional **note** for the customer.
3. Click **Send quote**. The customer is messaged a **private pay link** (email first, then SMS). The link is also shown on the page so you can send it yourself, for example on WhatsApp. **Change the quote** any time before the customer orders; the link keeps working.

Automatic quotes (settings at the top of Link requests)

Setting	What it does
Quote by itself when the price is read from the shop's own web page	On by default. The safe automatic option.
Also quote by itself from the price the customer typed	Off by default. The system cannot check this price.
Safety margin on customer-typed prices	Percentage added on top of the typed price (default 5%).
Automatic limit	Above this UK price for one item (default £150) a person must quote.
Hold each price for	Days a quote stays valid (default 3).
Item types and weights	The list customers choose from, with a default weight each, used to work out shipping. Keep a catch-all such as "Other or not sure" last.

What the customer does

They open their private link (or **See price and pay** in their account), sign in if needed, see the **full cost in cedis**, choose delivery and shipping, and pay through the normal checkout. The quote expires after the days you set; an expired link asks them to contact you.

Confirming a payment by hand

If money has really arrived but the order still says **Awaiting payment** (no gateway is set up yet, a gateway message was missed, or the customer paid by bank transfer), open the order and use **Confirm payment by hand**. It needs the *Confirm a payment by hand* right, which only the super admin and **Manager** have unless you give it to someone.

- Pick how it was paid, enter the **reference** (bank, Mobile Money or gateway transaction ID) and **why** you are doing it, and tick that you have seen the money arrive.
- The order becomes **Payment received** and the customer is messaged, exactly as with a gateway payment. Customers see only "Payment received"; the reference and reason are kept in the **Payments** box on the order and the **Activity log**, with your name.
- It works only on an order still awaiting payment, only once, and the same reference cannot release two orders. Any waiting gateway attempt on the order is retired.
- It cannot be undone from this page. If you confirmed the wrong order, cancel it and refund as usual.

After payment

It is an **ordinary order**: same statuses, tracking, messages, refunds and costs. A **yellow banner** on the order tells you how the price was found:

- *read from the shop's page by the system*,
- *typed by the customer plus a margin, so it is not verified*, or
- *quoted by the team*.

Always check the shop's price again before you buy. If it is now higher, contact the customer first; you can cancel and refund from the order page.

Amazon UK requests

Amazon does not allow shops to read its pages, so the system never opens an Amazon link. Instead:

- The search page offers customers a link to Amazon UK's own search; they pick the item there and send it back by pasting the link, by the phone's **Share** menu or by the bookmarklet on the **Order from Amazon UK** page (`/amazon`).
- The link is cleaned to `amazon.co.uk/dp/<ASIN>` . Amazon.com and other Amazon stores are refused with a message pointing to the UK link.
- The name and price in the request come from **the customer** (or their bookmarklet), not from Amazon. They are only a guide. Always open the link, confirm the real price, size/colour and stock, and quote it yourself, or let the customer-typed-price automatic quote apply if you have switched it on (it adds your safety margin).

Requests with no link

A customer can describe an item in words instead of pasting a link. Those requests show a **No link** tag with **Search Amazon UK** and **Search Google UK** buttons that search for the customer's words. Find the exact item, paste its link into **Link to the item you found** (Amazon UK links are tidied; other Amazon stores are ignored), enter the UK price and weight and send the quote. They are never priced automatically. Always check size, colour and stock before quoting.

Limits to be aware of

- Amazon-style pages can show several prices (other sellers, other sizes). A single read may pick the wrong one. Check a few real items before relying on automatic quotes for a shop.

- Item-type weights are estimates; a heavy or bulky item may cost more to ship than the rate card suggests.

6. Using the admin on a phone

On a phone the sidebar becomes a top bar with a **Menu** button. The menu lists every page you can open, plus **View the site**, **Signed in as…** and **Sign out**, and closes when you choose a page. Orders, customers and items appear as cards with labelled fields instead of wide tables. Long forms scroll down; nothing needs sideways scrolling.

7. Staff accounts and access

Only the **super admin** sees **Staff accounts**. Staff cannot create or change accounts and cannot give themselves more access; that right can never be granted.

Creating an account

Admin → Staff accounts → + Add a staff account: name, email (their sign-in), a **first password** (use **Suggest one**; give it to them yourself), and a **role**. They must choose their own password at first sign-in.

Roles

Picking a role ticks its rights. Changing any tick switches the role to **Custom**. Granting a *change* right always grants the matching *view* right.

Right	What it allows	Manager	Operations	Customer support	Catalogue editor	Finance	Read-only
See the dashboard (dashboard.view)	Sales, revenue and customer figures.	✔	✔			✔	✔
See orders (orders.view)	Order list and details, including customer names, phones and addresses.	✔	✔	✔		✔	✔
Update orders (orders.manage)	Change order status, add tracking, cancel and refund.	✔	✔				
Record costs and see margins (orders.costs)	What the team paid to buy and ship items, and the profit on each order.	✔				✔	
Download the orders file (orders.export)	Export orders with customer details as a spreadsheet.	✔	✔			✔	
See customers (customers.view)	Customer accounts, contact details and order history.	✔	✔	✔			✔
Manage customers (customers.manage)	Disable or enable accounts and create password-reset links.	✔					
Handle link requests (requests.manage)	Quote link requests, and set the automatic-quote rules.	✔	✔	✔			

Right	What it allows	Manager	Operations	Customer support	Catalogue editor	Finance	Read-only
Manage shops (shops.manage)	Add, edit and delete shops and their logos.	✔			✔		
Manage items (items.manage)	Add, edit and hide items and photos.	✔			✔		
Manage catalogue sources (sources.manage)	Feeds, Shopify, WooCommerce, eBay, Diffbot and file imports.	✔			✔		
Review imported items (import.review)	Approve or reject items waiting in Import review.	✔			✔		
Moderate reviews (reviews.manage)	Publish or hide customer reviews.	✔			✔		
Change prices and delivery (pricing.manage)	Service charge, exchange rate, shipping rates and delivery areas.	✔					
Change the look of the site (appearance.manage)	Colour themes.	✔					
Confirm a payment by hand (orders.confirm_payment)	Mark an unpaid order as paid when the money arrived outside the payment flow. Sensitive: it starts the buying.	✔					
Manage integrations and keys (integrations.manage)	Payment, SMS, email and other service keys. Very sensitive.	✔					
See messages sent (messages.view)	The log of messages sent to customers.	✔	✔	✔			
Manage messages (messages.manage)	Retry failed messages and choose which updates are sent.	✔					
See the activity log (audit.view)	Who changed what, and when.	✔				✔	

Legal pages: privacy, terms and data deletion

The shop has three public pages, linked from the footer: `/privacy` (privacy policy), `/terms` (terms of service) and `/data-deletion` (how to delete an account and what is kept). Facebook, Google and Apple ask for these addresses, for example `https://your-domain/privacy`. Facebook's "data deletion instructions URL" is `/data-deletion`.

- They take your business name and contact details from **Pricing → Site**, so nothing about the business is written into the text.
- The wording describes what the shop does today (what it collects, who it shares with, how deletion works). **It is a starting draft, not legal advice.** Have a lawyer in your market read it before you rely on it, check the governing law (it says Ghana), the refund wording, and whether you must register with the Data Protection Commission.
- If you change what the shop collects or who it shares data with, change the wording in `src/app/(site)/privacy/page.tsx` and update the date in `src/components/LegalPage.tsx`.

Managing an account

Open it from the list to: **change name, role and rights** (this signs them out so the new rights apply at once), **switch off or on** (switching off signs them out everywhere immediately), **reset the password** (they must choose a new one; they are signed out), or **delete** (confirmation needed; their past actions stay in the activity log under their name).

What staff see

The menu shows only the pages they may open. Typing the address of another page shows **No access**. On an order, staff who can view but not update see "You can look at this order but not change it", with no status or tracking buttons; costs and margins are hidden unless they have that right; the orders download needs its own right.

Choosing roles well

- **Customer support** and **Operations** are safe for most people.
- **Manager** includes **integration keys** and **prices**. Give it to very few people.
- Anyone who handles payments needs **Finance** or **Operations**, not **Read-only**.

8. Good habits

- Give each person their **own** account. Never share the super admin password.
- Switch off the account of anyone who leaves **the same day**.
- Check the **Activity log** weekly.
- Keep **backups** (see Deployment and operations).
- Before you buy a link order, **check the price again**.
- Treat imported prices as a guide: read the price on the shop before buying.

3. Customer guide

Written for shoppers, in plain language. Staff can copy any part of it into a help page, a WhatsApp reply or a message to a customer.

What this shop does

You choose items from UK shops. You pay us **once, in Ghana cedis**. We buy the items in the UK, ship them to Ghana, and a courier brings them to your address. You can follow every step.

The price you see includes the items, our service charge, shipping to Ghana and delivery to your address. **Customs duty or taxes charged in Ghana, if any, are not included.**

Finding something

- **Browse**: the shop pages, the departments (Fashion, Electronics and so on) and the **Browse** menu at the top.
- **Search**: type in the search bar. Suggestions appear as you type; filters and sorting are on the results page.
- **Prices**: every price is shown in **pounds** (the UK price) and **cedis** (what you pay). On a product page, the **"to your door"** box shows the full cost for your delivery area.
- **Saved items**: tap the heart to save something for later (you need to be signed in).

Can't find it here? Paste the link

You can buy from **almost any UK website**, even if it is not listed here.

1. Open the product on the UK shop's website and choose your size and colour.
2. Copy the web address from your browser.
3. Paste it into the box on our home page (or the **Add by link** button at the top, the search bar, or the **Paste link** tab on a phone).
4. We show the price if we can read it. Choose the quantity, the kind of item and your size or colour, then tap **Get my full price and pay**.
5. If we cannot price it straight away, we check it for you and **send you a message** with your price and a link to pay. It also appears in your account under **Your link requests**.

The price is held for a few days. After that it may have changed, so ask us for a fresh one.

No link? Just tell us what you want

Open **Request any item**, type what you are looking for (for example "OnePlus 13 256GB, black") and send it. You do not need a link. Our team finds the item on Amazon UK or another UK shop, checks the price and stock, and messages you the full cost in cedis with a link to pay.

Ordering from Amazon UK

We do not show Amazon's products in our own search, but ordering from **Amazon UK** is just as easy:

1. Search for the item here. Under the results, tap **Search "..." on Amazon UK**; Amazon opens in a new tab.
2. On Amazon, open the item you like and choose your size, colour or storage.
3. Bring it back to us in whichever way is easiest:

- **On a phone with the Amazon app:** tap **Share**, then choose our shop in the list (install our shop on your phone first from the **Order from Amazon UK** page).
- **On a computer or phone browser:** copy the address and paste it into the box, or use the one-click button from the **Order from Amazon UK** page (link at the bottom of every page).

4. Check the price on Amazon and type it in. We add our charges and show the full cost in cedis.

Only **Amazon UK** (amazon.co.uk) links work, because we buy from UK shops. If you paste an amazon.com link we will say so and suggest the UK one. The price you type is a guide: we check it on Amazon before buying and will contact you if it is different.

Ordering

1. **Add to cart** (choose your size and colour first where there is a choice).
2. **Checkout**. You need an account: your name, a phone number we can reach you on, and a password.
3. Choose your **delivery area** and **shipping method** (for example Air freight or Express air). The total updates as you choose.
4. Enter your delivery address and a landmark or GhanaPost GPS code if you have one.
5. Choose how you want **updates**: SMS, email and/or WhatsApp.
6. **Place order and pay.** You are sent to the payment page of our payment partner. We never see or store your card details.

There is a **minimum order** (items only). The cart tells you how close you are.

Paying

Payment options depend on what the shop has switched on, for example Mobile Money and cards through Paystack or Flutterwave, or international cards through Stripe. Your total is always shown in cedis. A card payment through Stripe is charged in pounds: the same amount converted at the rate shown on your order (your bank may add its own currency fee). Your order is only bought **after payment is confirmed**. If a payment fails you can try again from your account.

Tracking your order

Open **Account → Orders**, or use **Track an order** with your order number and the phone number or email you used.

Your order moves through these steps:

Step	What it means
Awaiting payment	We start buying as soon as your payment is confirmed.
Payment received	Your payment is confirmed and your items are queued for buying.
Buying from the UK shop	Our team is placing your order with the UK shop.
Bought from the UK shop	The UK shop has accepted the order and will send it to us.
Received at our UK address	Your parcel has reached our UK address and is being prepared for export.
On its way to Ghana	Your parcel has left the UK.
Clearing customs in Ghana	Your parcel has arrived in Ghana and is being cleared.

Step	What it means
Out for delivery	A rider is bringing your order to your address.
Delivered	Your order has been delivered.

Where we have a carrier and reference for a stage you will see them, often with a link. **Account → Updates** is a feed of everything that has changed.

Cancelling and refunds

You can ask us to cancel **before we have bought your items** (up to the *Buying from the UK shop* step). Once an item is bought from the UK shop it usually cannot be cancelled. If you paid and the order is cancelled, we refund you and mark the order **Refunded**; the time to reach your account depends on your payment method. Contact us to ask.

Your account

Sign in with Google, Facebook or Apple. If the shop offers it, you will see these buttons on the sign-in and sign-up pages. The first time, we ask for your phone number (we arrange deliveries by phone) and then your account is ready. If you already have an account with the same email, Google or Apple signs you in to it. If it is Facebook, or the details do not match, sign in with your password first, then connect it from **Account → Security → Connected sign-in**. If you used one of these buttons you do not have a password; choose one with **Forgot your password** if you want to sign in the other way too.

Page	What it is for
Overview	Your latest order and its tracking, quick counts, your link requests and "Buy again".
Orders	Every order, with tracking.
Updates	A feed of changes to your orders.
Saved items	Your wishlist.
Addresses	Up to 10 saved delivery addresses.
Profile	Your name, phone, email and how you want to be notified.
Security	Change your password, sign out other devices, or delete your account.

Forgotten password? Use **Forgot password** on the sign-in page. We send a link that works for 60 minutes. If you cannot receive it, contact us and we can create a one-time link for you.

Your information

Our privacy policy says what we collect and why, and our terms of service set out how the service works. To delete your account, see Delete your data (Account → Security → Delete my account). Orders are kept for our records after deletion, without a link to an account.

Reviews

Once an item is **delivered** you can review it, once. Your review shows your first name and the initial of your last name (for example "Ama M."). We may hide reviews that break our rules.

Staying safe

- We will **never ask for your password** or card number by message, phone or WhatsApp.
- Pay only on the payment page you reach from our site.
- Messages from us come from our shop's name; if in doubt, sign in to your account and check your orders there.

Getting help

Use the contact details shown on the site (for example the WhatsApp number in the footer) and quote your **order number**.

4. Catalogue sources

How products get onto the site, and the rules the importer follows. Everything here is under **Admin → Catalogue sources** (needs the *Manage catalogue sources* right; approving imports needs *Review imported items*).

The idea

A **source** is one place you are allowed to read a shop's products from. Each source belongs to one of your **shops** and has its own schedule. When it runs, it **reads**, **checks**, then either **publishes** automatically or holds items for **review**.

A new database starts with one shop, **eBay UK**, and nothing else. Nothing fictional is created in production.

Choosing the right kind

Kind	Best for	What you give it	Notes
Product feed (CSV or JSON)	Official and affiliate feeds (Awin, CJ, Rakuten, Impact…)	The feed address	The most reliable: prices, stock and images come from the shop. Columns are recognised automatically; override with lines like `price=cost.gbp`.
eBay (official API)	Real UK listings with photos	Free eBay developer keys (Admin → Integrations → Catalogue APIs) and your searches, one per line (up to 10)	Only new, fixed-price, UK-located listings priced in pounds. Search results vary, so a missing listing is never treated as removed.
Diffbot (Product API)	Product pages you list, when you hold or are allowed the data	Your Diffbot token (Admin → Integrations → Catalogue APIs) and the product page addresses, one per line	Paid: one Diffbot credit per product on every run. Diffbot returns name, price, was-price, stock, brand and photo. Only prices in pounds are used. The shop's `robots.txt` is checked first and a disallowed page is skipped without calling Diffbot. The shop's terms still apply because Diffbot reads the page for you.
WooCommerce shop	Small UK shops on WooCommerce	The shop's main address, for example `https://shop.co.uk`	Reads the public product list (`/wp-json/wc/store/v1/products`) only if the shop's `robots.txt` allows it and the shop prices in pounds. Brings sizes and colours across as choices. Products sold on another website, grouped products, and products whose sizes cost different amounts are skipped. Weight is not published, so the source's default weight is used. Get the owner's agreement first.
Shopify shop	Small UK brands on Shopify	The shop's main address, for example `https://brand.co.uk`	Reads the public product list only if the shop's `robots.txt` allows it and the shop prices in pounds. Brings sizes and colours across as choices. Products whose sizes cost different amounts are skipped (the site holds one price per product). Get the owner's agreement first.
Shop website (sitemap + pages)	Shops whose terms and `robots.txt` allow it	The sitemap address	Reads product data (JSON-LD or Open Graph) from each page, one at a time, slowly.
File import (CSV or JSON)	Data you collected yourself, for example a spreadsheet	Create the source, then upload the file on its page	Nothing is fetched from any shop. Prices must be in pounds. Re-uploading the same file updates prices and stock. A file never removes products. Items disappear after the "hide items not refreshed" days unless you upload again.

Kind	Best for	What you give it	Notes
Pasted links	One-off items	Up to 20 links at a time	Prices are re-checked automatically.

How to tell if a shop's list is readable on WooCommerce

Open `theirshop/wp-json/wc/store/v1/products` in a browser. A readable shop shows a long block of text starting with `[{"id":` and containing `"currency_code":"GBP"`. A login page, an error, or `rest_no_route` means the list is switched off or the shop is not on WooCommerce, so it cannot be used.

How to tell if a shop is on Shopify

Open `theirshop/meta.json` in a browser. A Shopify shop shows text containing `"currency":"GBP"`; any other site shows an error page.

Large retailers

Many large UK retailers (for example Amazon, Argos, Tesco, Asda, Currys, Next, M&S, B&Q) **forbid or block** automated reading, and that includes a service such as Diffbot reading their pages for you. The importer will stop if they refuse. For them use an **affiliate feed**, or handle their products as **link orders** (Admin guide → Link orders).

Setting up a source

1. **Add a source**, choose the **shop** the items belong to and the **kind**.
2. Enter the address (or searches), and optionally **column names** for feeds.
3. Tick the **permission box**: you confirm you have checked the shop's terms or hold a licence for the data. **A source cannot be switched on without it.**
4. Click **Check this setup first**. It shows a few items and any problems without saving anything.
5. **Create source**, then **Run now** (or wait for the schedule).

Settings on a source

Setting	Meaning	Default
Switch on	Runs on its schedule	on when created
Publish new items automatically	New items go live without review	on
Update prices and stock automatically	Changes to live items apply by themselves	on
Hold a price move bigger than (%)	Bigger jumps wait in Import review	40
Run every (hours)	How often it runs	24
Hide items not refreshed for (days)	So an old price never stays on sale	14
Items per search / Pages / Products per run	Size of each run (up to 500)	50
Seconds between page requests	At least 2; a longer `Crawl-delay` in `robots.txt` always wins	3
Category and weight when the shop gives none	Used for shipping	weight 500 g

What runs by itself

The scheduler inside the server wakes every **ten minutes** and runs each source that is on and due. It then hides items that have not been refreshed for too long. To run it from your own scheduler instead, set `INGEST_AUTORUN=false` and call `GET /api/cron/ingest` with the cron secret.

For each item it reads:

- **New** → published (unless auto-publish is off or a check fails).
- **Changed price, stock or photo** → the live product is updated (unless it fails a check, then it is **held**).
- **Disappeared from a complete feed** → hidden. This happens only for complete feeds, and only if at least half of the previous items are still there (so a half-empty feed cannot wipe your shop).
- **Not refreshed within the stale window** → hidden.

Checks that send an item to review

- Price under **50p** or over **£10,000**.
- A name that looks like a warranty, protection plan, insurance or fee (for example "Worry-Free Purchase"): these are add-ons, not products.
- Name shorter than 3 characters, or no link back to the shop.
- A price move bigger than the source's limit.
- Any update, if you turned automatic updates off.

Rules the importer follows (and will not break)

- It identifies itself as **ShopCatalogBot**, with the address of this site in its user agent. (There is no explanation page; add one if you want shops to be able to read about it.)
- It obeys `robots.txt` and any `Crawl-delay`, and waits at least two seconds between requests to one shop.
- It never reaches private or internal addresses, and only uses ports 80 and 443.
- When a shop answers **401, 403, 429, 451**, or shows a verification page, it **stops**, **pauses that source for 24 hours**, and tells you. It **never** retries with another identity, rotates addresses, or tries to get past CAPTCHAs or blocks. Switching the source off and on again clears the pause.
- Only **prices in pounds** are accepted.
- Feed addresses (which often contain keys) are stored **encrypted**.
- Only one run per source at a time (a lock expires after 30 minutes).
- Images are **linked** from the source, not copied. Confirm your licence covers showing them.

Removing a source

Open the source → **Remove this source**. Choose **keep its products** (they stay on the shop but stop being refreshed, so their prices can go out of date) or **remove its products** (they disappear from the shop, carts and wishlists). Orders already placed keep their own record. A source with a run in progress cannot be removed.

Troubleshooting a source

You see	Likely cause	What to do
"Blocked by the shop… HTTP 403"	The shop refuses automated reading	Do not retry. Use an affiliate feed, or link orders. Remove or switch off the source.
"Add your eBay App ID and Cert ID…"	eBay keys are missing or eBay is off	Enter them under Integrations → Catalogue APIs and switch eBay on.
"That address does not look like a WooCommerce shop, or its product list is switched off"	Not WooCommerce, or the shop turned the public list off	Try the `/wp-json/wc/store/v1/products` test above.
"That address does not look like a Shopify shop"	Not a Shopify shop (no `/meta.json`)	Check the address; try the `/meta.json` test above.
"This shop prices in USD, not pounds…"	The shop's currency is not GBP	It cannot be used.
Many items "skipped: variants have different prices"	Sizes cost different amounts	Those products are skipped by design.
Items keep going to Import review	Prices look odd or jump a lot	Check the feed; raise the limit only if the moves are real.
Items disappear after two weeks	The stale window passed	Make sure the source is on and running, or lengthen the window.

5. Deployment and operations

For the developer who runs the server. The step-by-step deployment recipes are also in `DEPLOY.md` at the top of the repository; this document adds the reasoning, the full settings reference and the runbooks.

5.1 What the server needs

The shop is **one Node.js server** that keeps its data in **one SQLite file** and runs its background jobs **inside the same process**. So the host must provide:

Requirement	Why
A **persistent disk**	The database file, and uploaded photos and logos (stored in an `uploads` folder next to the database file), live on it.
Exactly one always-running instance	SQLite allows one writer, sign-in throttles are kept in memory, and the catalogue scheduler and message retries run inside the server. Two instances would fight over the file and double-run jobs.
HTTPS	Cookies are marked `Secure` in production, and payment gateways require HTTPS return and webhook addresses.
Node.js 22 (supplied by the Docker image)	The app targets Node 22.

Serverless hosts (Vercel, Netlify, Cloudflare Workers) **will not work**: no persistent disk and no always-on process. Render, Fly.io, Railway or any VPS with Docker all work.

5.2 The current deployment (Render)

Item	Value
Host	Render, **Web Service**, Docker runtime
Service name	`ukgh-shop` (from `render.yaml`; keep this name, because renaming a Blueprint service makes Render create a new one without the disk)
Region	**Frankfurt** (closest to Ghana and the UK). A region cannot be changed after the service exists.
Plan	**Starter** (a persistent disk needs a paid instance)
Disk	`shopdata`, **1 GB**, mounted at `/data`
Database file	`/data/shop.db` (`DATABASE_PATH`)
Health check	`/api/health`
Code	GitHub repository `Princegates/Ecommerce-UK-Shopping`, branch deployed from Render
Address	`https://ukgh-shop.onrender.com` until **yenkoexpress.com** is connected (see 5.4, Your own domain)

Updating the live site: push code to the branch Render watches (or press **Manual Deploy** in Render). The Docker image rebuilds and the server restarts. Database changes are applied automatically at start. Expect a short interruption while it restarts.

Container start-up: `docker-entrypoint.sh` makes the data folder writable for the app user and then drops root privileges (user id 10001) before starting the server.

5.3 Settings reference

Set these as **environment variables** on the host (Render → service → **Environment**). Anything you leave out of the environment can still be saved in **Admin → Integrations** for provider keys; **an environment value always wins** over a saved one.

Required in production

Variable	Purpose
`APP_URL`	The public address, `https://…`, no trailing slash. Used for payment return links, webhook addresses and the links in messages. It is **never** read from request headers. Without an `https` value, customers cannot be messaged their link-order pay links, and the Google, Facebook and Apple sign-in buttons stay hidden.
`ADMIN_PASSWORD`	The **super admin** password. A long passphrase. **Only the developer should know it.**
`ADMIN_SECRET`	16 or more random characters. Signs admin sessions. Changing it signs everyone out of the admin. **If you have not set `SETTINGS_ENCRYPTION_KEY`, the saved keys are encrypted with this value, so changing it would also make them unreadable. Always set a separate `SETTINGS_ENCRYPTION_KEY`.**
`SETTINGS_ENCRYPTION_KEY`	16 or more random characters. Encrypts provider keys and feed addresses saved in the database. **If you lose or change it, saved keys become unreadable** and must be entered again. Falls back to `ADMIN_SECRET` if unset.
`CRON_SECRET`	16 or more random characters. Protects the `/api/cron/*` endpoints (they answer 404 until it is set).
`DATABASE_PATH`	The database file, e.g. `/data/shop.db` (the Docker image sets this).

Generate random values with `openssl rand -base64 32`.

Optional

Variable	Purpose	Default
`ALLOW_DEMO_PAYMENTS`	Enables the pretend payment page. **Leave unset in production.**	off in production
`INGEST_AUTORUN`	`false` turns off the built-in catalogue scheduler (use `/api/cron/ingest` instead).	on
`SEED_SAMPLE_DATA`	`true` creates made-up sample shops on a **new** database. **Development only.**	off
`UPLOAD_DIR`	Where uploaded photos are stored.	`uploads` next to the database

Provider keys (environment or Admin → Integrations)

Provider	Variables
Stripe	`STRIPE_SECRET_KEY`, `STRIPE_WEBHOOK_SECRET`, `STRIPE_CHARGE_CURRENCY` (GBP or GHS)
Paystack	`PAYSTACK_SECRET_KEY`
Flutterwave	`FLUTTERWAVE_SECRET_KEY`, `FLUTTERWAVE_SECRET_HASH`
Arkesel (SMS)	`ARKESEL_API_KEY`, `ARKESEL_SENDER_ID`

Provider	Variables
Twilio (SMS and WhatsApp)	`TWILIO_ACCOUNT_SID`, `TWILIO_AUTH_TOKEN`, `TWILIO_SMS_FROM`, `TWILIO_WHATSAPP_FROM`, `TWILIO_WHATSAPP_CONTENT_SID`
Meta WhatsApp Cloud API	`WHATSAPP_ACCESS_TOKEN`, `WHATSAPP_PHONE_NUMBER_ID`, `WHATSAPP_TEMPLATE_NAME`, `WHATSAPP_TEMPLATE_LANGUAGE`, `WHATSAPP_GRAPH_VERSION` (default v22.0)
Resend (email)	`RESEND_API_KEY`, `RESEND_FROM`
Postmark (email)	`POSTMARK_SERVER_TOKEN`, `POSTMARK_FROM`, `POSTMARK_MESSAGE_STREAM` (default outbound)
eBay	`EBAY_APP_ID`, `EBAY_CERT_ID`, `EBAY_ENVIRONMENT` (production or sandbox)
Diffbot	`DIFFBOT_TOKEN`
Sign in with Google	`GOOGLE_CLIENT_ID`, `GOOGLE_CLIENT_SECRET`
Sign in with Facebook	`FACEBOOK_APP_ID`, `FACEBOOK_APP_SECRET`
Sign in with Apple	`APPLE_CLIENT_ID` (the Services ID), `APPLE_TEAM_ID`, `APPLE_KEY_ID`, `APPLE_PRIVATE_KEY` (the .p8 contents; `\n` or one line is fine)
ExchangeRate-API	`EXCHANGERATE_API_KEY`
Open Exchange Rates	`OPENEXCHANGERATES_APP_ID`

Details of each provider are in Integrations.

5.4 First deployment, step by step

1. Prepare the settings in 5.3.
2. Create the service (Render: **New → Blueprint**, select the repository; `render.yaml` creates the service and disk).
3. Open `https://<address>/api/health`. It should answer `{"ok":true}`.
4. Sign in as the super admin at `/admin/login?developer=1`.
5. Work through the first-day checklist.
6. Register each payment gateway's **webhook address** (shown on its card) and press **Test**.
7. Place a real small order end to end before announcing the shop.

Other hosts

- **Any server with Docker:** `cp .env.example .env.production`, fill it in, then `DOMAIN=shop.example.com docker compose up -d --build`. Caddy fetches and renews the HTTPS certificate. Data is in the `shopdata` volume.
- **Fly.io:** `fly launch --no-deploy --copy-config`, `fly volumes create shopdata --size 1 --region lhr`, set the secrets, `fly deploy`, `fly certs add <domain>`. `fly.toml` keeps one machine always on.

Your own domain

Add the domain in the host (Render → service → **Settings → Custom Domains**), create the DNS records it shows, wait for the certificate, then change `APP_URL` to the new address and **update every gateway's webhook and return address** to match, and the redirect address of Google, Facebook and Apple sign-in.

Connecting yenkoexpress.com, in order:

1. Render → service → **Settings → Custom Domains**: add `yenkoexpress.com` and `www.yenkoexpress.com`. Create the DNS records Render shows at the domain registrar (usually an `A` record for the bare domain and a `CNAME` for `www`), then wait for the certificate.
2. Change `APP_URL` to `https://yenkoexpress.com` and redeploy.
3. Update the return and webhook address in Stripe, Paystack and Flutterwave (`https://yenkoexpress.com/api/webhooks/...`).
4. In the Google, Facebook and Apple consoles, change the sign-in redirect address, the privacy policy address (`/privacy`) and the terms address to the new domain.
5. Verify `yenkoexpress.com` in Resend (add the SPF and DKIM records it shows) and set the sender to `YenkoExpress <orders@yenkoexpress.com>` in **Admin → Integrations**.
6. Set the SMS sender name in Arkesel (11 characters at most, for example `YENKO`) and have it approved.
7. Update the site address in Facebook and Google app settings, and in the eBay developer account if it lists one.
8. Keep the `onrender.com` address working for a few days, and add a redirect from `www` to the bare domain at the registrar or in Render.

The site name shown to customers is **YenkoExpress** (Admin → Prices and settings → Site name). Existing installs that still had the old default name are renamed automatically when the server starts.

5.5 Scheduled jobs

Job	Runs	Notes
Catalogue sources	Built in, every 10 minutes (first run 1 minute after start)	Each source has its own interval (default 24 hours). To run elsewhere: `INGEST_AUTORUN=false` and call `GET /api/cron/ingest`.
Message retries	Messages are also sent right after an event; failed ones are retried	Call `GET /api/cron/messages` every few minutes from a scheduler for reliability.
Exchange-rate sync	Only if you use a rate feed	Call `GET /api/cron/fx` hourly or daily.

Call cron endpoints with `Authorization: Bearer $CRON_SECRET`, for example:

```
curl -H "Authorization: Bearer $CRON_SECRET" https://your-site/api/cron/messages
```

On Render you can use a **Cron Job** service for this.

5.6 Backups (set up before you take orders)

All data is in **one file**, which is in **WAL mode**. A plain copy of `shop.db` while the server runs can miss recent writes, so use SQLite's backup function, not `cp`.

Open a **Shell** on the Render service and run:

```
node -e "const D=require('better-sqlite3');new D(process.env.DATABASE_PATH).backup('/data/backup-
'+Date.now()+'.db').then(()=>{console.log('backup done');process.exit(0)})"
```

Then **copy the backup off the machine** (a backup that lives on the same disk is not a backup). Also copy the `/data/uploads` folder, which holds uploaded photos and logos.

Better options: Render **disk snapshots** (restorable from the dashboard; check Render's current documentation for how often they are taken and how long they are kept), or Litestream streaming the database to object storage continuously.

Test a restore at least once: stop the server, put a backup in place as `shop.db` (delete any `shop.db-wal` and `shop.db-shm`), start the server, and check you can sign in and see orders.

5.7 Monitoring

- **Health:** `GET /api/health` returns `{"ok":true}` if the server is up and can read the database, otherwise `503`. Point an uptime monitor (UptimeRobot, Better Stack) at it.
- **Logs:** Render → service → **Logs**. Lines starting `[ingest]` come from the catalogue scheduler; `[webhook:<provider>]` from payments; `[orders]`, `[audit]` and `[request]` from the matching areas. Secrets are never logged.
- **In the admin:** the Dashboard **Needs attention** list, **Messages** (failed sends), **Catalogue sources** (stopped sources) and **Integrations → Readiness**.
- **Disk:** the 1 GB disk holds the database and uploads. Check usage in Render → **Disks** now and then and enlarge it before it fills.

5.8 Runbooks

Redeploy or roll back

- **Redeploy:** Render → **Manual Deploy → Deploy latest commit**.
- **Roll back code:** Render → **Deploys** → pick an earlier deploy → **Rollback**. Database changes only ever move **forward** and are additive, so older code normally still runs, but test a rollback on a copy before relying on it. Take a backup before any risky deploy.

I lost the super admin password

Set a new `ADMIN_PASSWORD` in the host's environment and redeploy. Existing staff accounts are unaffected.

A staff member forgot their password

As the super admin: **Staff accounts → the person → Reset password**. They choose a new one at their next sign-in.

A staff member left

Switch their account off (it signs them out immediately), then delete it when convenient.

A key may have leaked

Rotate it at the provider, enter the new key in **Admin → Integrations** (or the environment), and press **Test**. If `ADMIN_SECRET` leaked, change it (this signs every admin out). If `SETTINGS_ENCRYPTION_KEY` leaked, change it and re-enter all saved provider keys.

Customers say they cannot pay

Check **Integrations → Readiness** (is a gateway configured and switched on?), the gateway's dashboard for errors, and the **Needs attention** list for "Payments with the wrong amount". Confirm `APP_URL` is the live `https` address and the webhook is registered.

Messages are not arriving

Messages shows failed sends and the reason. Check the provider key, sender name or template approval, and that the channel is switched on and the status is enabled under **Which updates are sent**. Press **Retry**.

A catalogue source stopped

Open the source: the message says why. See Catalogue sources → Troubleshooting.

The site is down

Check Render's status and **Logs**, and `/api/health`. A full disk is a common cause: free space or enlarge it. A restart (Manual Deploy) is safe.

5.9 Costs to expect

Item	Notes
Render Starter web service	Monthly, with the persistent disk (the disk is what keeps your data).
Payment gateway fees	Charged by Stripe, Paystack or Flutterwave per transaction.
SMS / WhatsApp / email	Per message, depending on the provider.
Domain name	Yearly, optional at first.
eBay, Shopify, WooCommerce reads	Free (eBay developer keys are free).
Diffbot reads	Paid: one Diffbot credit per product page read, on every run of the source.

Do not switch to a free plan or remove the disk: you would lose the database.

5.10 Not covered yet

Phone or email verification at sign-up, a shared rate-limit store for running several instances, a Content-Security-Policy header, two-step sign-in for staff, and an email invitation for new staff (the super admin gives the first password personally).

6. Architecture

Technology

Layer	Choice
Framework	**Next.js 16** (App Router) with **React 19** and **TypeScript**
Styling	**Tailwind CSS 4**, with the shop's own component classes in `src/app/globals.css` and colour themes set as CSS variables
Database	**SQLite** through `better-sqlite3` (synchronous, one file, WAL mode)
Validation	**zod**
Payments	Stripe SDK, plus direct calls for Paystack and Flutterwave
Tests	**Vitest** (unit and database tests) and **Playwright** (end-to-end browser run in `e2e/`, started with `npm run e2e`)
Fonts	Figtree and Bricolage Grotesque (bundled, no external requests)

The app is built as a **standalone server** (`output: "standalone"`) and shipped as a Docker image.

> `AGENTS.md` at the repository root warns that this Next.js version has breaking changes from older ones. Read the guides in `node_modules/next/dist/docs/` before changing framework-level code.

Folder map

```
src/
  app/
    (site)/            customer pages: home, shops, product, cart, checkout, pay, order, account, quote,
request, track, login…
    admin/             admin: login, server actions, export route
      (panel)/         every signed-in admin page (dashboard, orders, customers, requests, shops, items,
sources, import, reviews,
                       pricing, shipping, zones, appearance, integrations, messages, audit, users,
account, no-access)
    api/               cart summary, products, suggest, link-preview, health, cron/*, webhooks/[provider]
    actions/           customer server actions (account, cart, checkout, request, track, reviews,
wishlist, delivery)
    uploads/[name]/    serves uploaded photos and logos
    robots.ts, layout.tsx, globals.css
  components/          shared UI (storefront in the folder root and shop/, account/, admin/)
  lib/                 all the logic (see below)
    ingest/            catalogue importer: network safety, parsers, sources, run engine, scheduler
    payments/          gateway adapters and payment confirmation
    notify/            message templates, senders and the outbox
  instrumentation.ts   starts the catalogue scheduler when the server starts
docs/                  this documentation
```

The main modules in `src/lib`

Module	Responsibility
`schema.ts`, `db.ts`, `seed.ts`	Database definition, opening, migrations, first-run data.

Module	Responsibility
`pricing.ts`	**Pure** price calculation (no I/O) used by the product page, cart, checkout and order creation.
`money.ts` , `fx.ts` , `fx-api.ts`	Formatting and parsing of money; the exchange rate and its optional market feed.
`settings.ts`	Reads and writes site settings.
`catalog.ts` , `browse.ts`	Shops, products, departments, search and filtering.
`cart.ts`	Cart token cookie and cart lines.
`orders.ts` , `order-status.ts`	Quoting and creating orders, statuses and who may move them, tracking.
`link-orders.ts` , `link-auto.ts` , `link-submit.ts`	Link requests, quotes, automatic quoting, item types, and paying for a quote.
`customers.ts` , `customer-session.ts` , `password.ts`	Shopper accounts, sessions, password hashing (scrypt).
`admin-users.ts` , `permissions.ts` , `auth.ts`	Staff accounts, rights and roles, admin sessions and the access checks.
`audit.ts` , `analytics.ts` , `margin.ts`	Activity log, dashboard figures and attention list, order margin.
`integrations.ts` , `secrets.ts`	Provider registry and encrypted key storage.
`payments/*`	One adapter per gateway; webhooks and the return page are confirmed here.
`notify/*`	Message wording, providers and the retrying outbox.
`ingest/*`	The catalogue importer (below).
`uploads.ts`	Photo and logo storage with file-type checks.
`themes.ts` , `department-icons.ts`	15 colour themes; department icons.
`throttle.ts` , `cron-auth.ts` , `app-url.ts`	Rate limits, cron protection, the public address.

How a request flows

- **Pages** are **server components**. They read the database directly (it is local and synchronous) and render per request (`dynamic = "force-dynamic"`), so prices and stock are always current. Interactive bits (cart drawer, search suggestions, checkout form, link finder) are small client components.
- **Changes** go through **server actions** (`"use server"`). A customer action checks the shopper's session; an admin action calls `requirePermission(...)` first. Actions validate input with zod, call the `lib` function, then redirect with a `?saved=1` or `?error=…` message.
- **Route handlers** serve things that are not pages: the payment return, webhooks, cron endpoints, the health check, search suggestions, link previews, uploaded files and the orders CSV.
- **Money rule:** the browser never decides a price. The server recomputes every amount from live data when an order is created, and checks the amount again with the gateway before marking it paid.

An order, end to end

1. **Cart** (`cart_items` , tied to a cookie) is priced by `quoteCart` using live settings.
2. **Checkout** (`placeOrderAction`) requires a signed-in customer, validates the details, and `createOrder` writes `orders` , `order_items` and the first `order_events` row inside one transaction, with status *Awaiting*

payment.

3. The customer is sent to **`/pay/<payment reference>`** and chooses a gateway. The gateway creates a hosted checkout and records a `payments` row.
4. The gateway returns the customer to **`/pay/<ref>/return`** and also calls **`/api/webhooks/<provider>`**. Both end in `applyOutcome`, which checks the **amount and currency**, applies each event **once** (`webhook_events`) and calls `markPaid`.
5. `markPaid` sets *Payment received* and queues messages in the **outbox**.
6. Staff move the order forward; each change writes `order_events` and may queue messages.

A **link order** differs only at the start: a `link_requests` row and a quote create the order through `placeLinkOrder`, which uses the same `createOrder`.

The catalogue importer (`src/lib/ingest`)

```
source (catalog_sources) ─► gather items ─► stage each item (import_items) ─► publish / update / hold ─► products
        ▲ scheduler (every 10 min) or "Run now"        │ checks: price range, name, link, price-move limit
```

- `net.ts`: **the only way the importer touches the network.** Validates addresses (http/https, ports 80/443, no credentials, no private hosts), resolves names safely, follows at most three redirects (re-checked each time), obeys `robots.txt`, waits between requests, and treats 401/403/429/451 or a challenge page as **stop**.
- `parse.ts`: price parsing (pounds only), feed column mapping, sitemap and product-page reading (JSON-LD, then Open Graph), title tidying.
- `ebay.ts`, `diffbot.ts`, `shopify.ts`, `woocommerce.ts`: the API-style sources.
- `run.ts`: staging, publishing, the price-move guard, removal rules, stale sweep, file import, link import, previews.
- `store.ts`: sources and their settings, with encrypted addresses.
- `scheduler.ts`: the in-process timer, started from `instrumentation.ts`.

Background work

There is no separate worker. Two timers live in the server process: the **catalogue scheduler** and the **message outbox** (which is also kicked right after an event and can be driven by `/api/cron/messages`). This is why **only one instance** may run.

Access control in the code

- `auth.ts` reads the signed session cookie once per request (`getAdmin`) and returns who is acting: the super admin, or a staff account whose status and session version are checked **against the database every time**.
- `requireAdmin()` signs people in; `requirePermission("<right>")` also checks the right and redirects to **No access**; `requireSuper()` is for staff management only.
- A test (`admin-guards.test.ts`) scans the source and fails if any admin page or action lacks a check. See Security.

Design decisions worth knowing

- **SQLite, not a hosted database:** simplest to run and back up for one shop; the cost is the single-instance rule.
- **Pure pricing module:** the same code prices everywhere, so what customers see is what they pay.
- **Orders are snapshots:** old orders never change when settings or catalogue change.
- **The importer refuses to evade blocks:** by design, so the business is never built on circumventing a shop's protections.
- **Rights are explicit and checked per action,** not just hidden in the menu.

7. Data model

All data is in **one SQLite database file** (`shop.db` , in WAL mode, foreign keys on). The schema is created in `src/lib/schema.ts` when the server starts; later additions are applied by `migrate()` in `src/lib/db.ts` (adds columns, rebuilds a table when a rule has to change). Nothing is ever dropped automatically.

Conventions:

- **Money is stored as whole numbers**: pence for pounds (`*_gbp_minor` , `price_minor`) and pesewas for cedis (`*_ghs_minor` , or `*_minor` on orders). No decimals.
- **Times** are UTC text in the form `YYYY-MM-DD HH:MM:SS` .
- **Secrets are encrypted** before they are stored (provider keys, feed addresses). Passwords and session tokens are **hashed**.

How the tables relate

```
shops ──< products ──< cart_items >── carts
  │           │
  │           └──< wishlist_items >── customers ──< customer_addresses
  │           └──< reviews              │  ├──< customer_sessions
  └──< catalog_sources                   │  └──< password_resets
          ├──< import_items ──> products (the live product it feeds)
          └──< import_runs
orders ──< order_items            (product_id is a plain number, not linked: an order keeps its own
record)
   ├──< order_events   ├──< order_tracking   ├── order_costs   └──< payments
link_requests ──> orders          (order_id once the customer has paid)
messages (the outbox)             webhook_events (payment events already applied)
settings, integration_settings, shipping_methods, delivery_zones, fx_rate_history
admin_users, audit_log
```

Important choices:

- **Orders are self-contained.** An order stores the customer's name, address, area name and fee, shipping method, every amount, and each item's name, shop name, price and weight **as they were**. Deleting a shop, product or delivery area never changes an old order.
- **A link order's items have no product.** `order_items.product_id` is empty for them; `source_url` holds the link.
- **`link_requests.order_id`** ties a request to the order it became.
- **The super admin is not in the database.** `admin_users` holds staff only.

What each table is for

Table	Purpose
`settings`	Key/value site settings: site name, exchange rate and markup, service charge rule, minimum order, support WhatsApp, theme, exchange-rate policy, automatic-quote rules, item types, notification rules.
`shops`	The shops shown on the site. `category` is the department. `logo_url` is an uploaded file or a link.
`products`	Items for sale: price, weight, size/colour options (JSON), photo, source link, active flag, last-synced time.
`carts` , `cart_items`	A visitor's cart, found by a cookie token.

Table	Purpose
`shipping_methods`	Shipping methods and their rate cards (JSON).
`delivery_zones`	Delivery areas and their fees.
`orders`	One row per order: status, payment status, payment reference, customer and delivery details, rate used, every amount, chargeable grams, notification choices, owning customer.
`order_items`	The lines of an order, as bought.
`order_events`	The history of an order: each status change with the note shown to the customer.
`order_tracking`	Tracking entries (stage, carrier, reference, link, note).
`order_costs`	What the order really cost (for margin). One row per order.
`payments`	Each payment attempt with a gateway: provider, reference, currency, amount, status.
`webhook_events`	Gateway events already applied, so a repeated webhook is ignored.
`link_requests`	Links customers sent: item, quantity, details, who sent it, status, and the quote (price, weight, note, expiry, private token, how the price was found) and the order it became.
`customers`	Shopper accounts: name, phone (international form), optional email, password hash, status, notification preferences, default area.
`customer_sessions`	Signed-in sessions (the cookie value is stored hashed).
`customer_identities`	Which Google, Facebook or Apple account is connected to which customer (the provider's own id, and the email it shared).
`oauth_states`	Sign-in attempts in progress: single use, 10 minutes, only hashes of the state and browser tie are kept.
`social_signups`	A first-time provider sign-in waiting for a phone number (30 minutes).
`customer_addresses`	Saved delivery addresses (up to 10 each).
`password_resets`	One-time reset links (stored hashed, valid 60 minutes).
`wishlist_items`	Saved items per customer.
`reviews`	Customer reviews (published or hidden).
`integration_settings`	Provider keys and switches. Secret values are encrypted.
`messages`	The outbox of SMS, WhatsApp and email messages: status, attempts, errors.
`admin_users`	Staff accounts: email, name, password hash, role, rights (JSON), status, must-change-password flag, session version, last sign-in.
`audit_log`	Who did what and when (actor, action, target, detail).
`fx_rate_history`	Every change to the exchange rate and why.
`catalog_sources`	Catalogue sources: kind, encrypted address, field map, schedule, rules, terms confirmation, last run result, pause time.

Table	Purpose
`import_items`	Everything the importer has read, with its decision (pending, published, held, rejected), fingerprint of price/stock/photo, options and the live product it feeds.
`import_runs`	One row per source run: counts and the message shown to you.

Columns

Generated from the database definition.

settings

Column	Type	Notes
`key`	TEXT	primary key
`value`	TEXT	required

shops

Column	Type	Notes
`id`	INTEGER	primary key
`slug`	TEXT	required
`name`	TEXT	required
`tagline`	TEXT	required; default ''
`category`	TEXT	required
`website_url`	TEXT	required; default ''
`description`	TEXT	required; default ''
`accent`	TEXT	required; default '#0b5d3b'
`logo_url`	TEXT	required; default ''
`notes`	TEXT	required; default ''
`active`	INTEGER	required; default 1
`sort`	INTEGER	required; default 0

products

Column	Type	Notes
`id`	INTEGER	primary key
`shop_id`	INTEGER	required; → shops
`slug`	TEXT	required
`name`	TEXT	required
`brand`	TEXT	required; default ''
`category`	TEXT	required; default ''

Column	Type	Notes
`description`	TEXT	required; default ''
`price_minor`	INTEGER	required
`weight_grams`	INTEGER	required; default 500
`options`	TEXT	required; default '[]'
`image_url`	TEXT	
`source_url`	TEXT	required; default ''
`active`	INTEGER	required; default 1
`created_at`	TEXT	required; default datetime('now')
`compare_at_minor`	INTEGER	
`deal_ends_at`	TEXT	
`last_synced_at`	TEXT	

carts

Column	Type	Notes
`token`	TEXT	primary key
`created_at`	TEXT	required; default datetime('now')
`updated_at`	TEXT	required; default datetime('now')

cart_items

Column	Type	Notes
`id`	INTEGER	primary key
`cart_token`	TEXT	required; → carts (on delete cascade)
`product_id`	INTEGER	required; → products
`quantity`	INTEGER	required
`options`	TEXT	required; default '{}'

shipping_methods

Column	Type	Notes
`id`	INTEGER	primary key
`code`	TEXT	required
`name`	TEXT	required
`eta`	TEXT	required; default ''
`rate_card`	TEXT	required
`active`	INTEGER	required; default 1
`sort`	INTEGER	required; default 0

delivery_zones

Column	Type	Notes
id	INTEGER	primary key
name	TEXT	required
areas	TEXT	required; default ''
fee_minor	INTEGER	required
eta	TEXT	required; default ''
active	INTEGER	required; default 1
sort	INTEGER	required; default 0

orders

Column	Type	Notes
id	INTEGER	primary key
number	TEXT	required
status	TEXT	required
payment_status	TEXT	required
payment_ref	TEXT	required
customer_name	TEXT	required
phone	TEXT	required
email	TEXT	required; default ''
zone_id	INTEGER	
zone_name	TEXT	required
address	TEXT	required
landmark	TEXT	required; default ''
notes	TEXT	required; default ''
shipping_code	TEXT	required
shipping_name	TEXT	required
fx_rate	REAL	required
fx_markup_pct	REAL	required
items_gbp_minor	INTEGER	required
items_ghs_minor	INTEGER	required
service_fee_minor	INTEGER	required
shipping_minor	INTEGER	required
delivery_minor	INTEGER	required
total_minor	INTEGER	required
chargeable_grams	INTEGER	required

Column	Type	Notes
`created_at`	TEXT	required; default datetime('now')
`notify_sms`	INTEGER	required; default 1
`notify_email`	INTEGER	required; default 1
`notify_whatsapp`	INTEGER	required; default 0
`customer_id`	INTEGER	→ customers (on delete set null)

order_items

Column	Type	Notes
`id`	INTEGER	primary key
`order_id`	INTEGER	required; → orders (on delete cascade)
`product_id`	INTEGER	
`shop_name`	TEXT	required
`name`	TEXT	required
`options`	TEXT	required; default '{}'
`quantity`	INTEGER	required
`unit_price_minor`	INTEGER	required
`weight_grams`	INTEGER	required
`line_ghs_minor`	INTEGER	required
`source_url`	TEXT	required; default ''

order_events

Column	Type	Notes
`id`	INTEGER	primary key
`order_id`	INTEGER	required; → orders (on delete cascade)
`status`	TEXT	required
`note`	TEXT	required; default ''
`created_at`	TEXT	required; default datetime('now')

link_requests

Column	Type	Notes
`id`	INTEGER	primary key
`url`	TEXT	required
`title`	TEXT	required; default ''
`details`	TEXT	required; default ''
`quantity`	INTEGER	required; default 1

Column	Type	Notes
price_seen	TEXT	required; default ''
name	TEXT	required
phone	TEXT	required
email	TEXT	required; default ''
status	TEXT	required; default 'NEW'
admin_note	TEXT	required; default ''
created_at	TEXT	required; default datetime('now')
token	TEXT	
quote_price_minor	INTEGER	
quote_weight_grams	INTEGER	
quote_note	TEXT	required; default ''
quoted_at	TEXT	
quote_expires_at	TEXT	
order_id	INTEGER	
item_type	TEXT	required; default ''
quote_source	TEXT	required; default ''
quote_basis_minor	INTEGER	
customer_id	INTEGER	

integration_settings

Column	Type	Notes
provider	TEXT	primary key
key	TEXT	primary key
value	TEXT	required
is_secret	INTEGER	required; default 0
updated_at	TEXT	required; default datetime('now')

payments

Column	Type	Notes
id	INTEGER	primary key
order_id	INTEGER	required; → orders (on delete cascade)
provider	TEXT	required
provider_ref	TEXT	required
attempt_ref	TEXT	required
status	TEXT	required; default 'PENDING'

Column	Type	Notes
`currency`	TEXT	required
`amount_minor`	INTEGER	required
`note`	TEXT	required; default ''
`created_at`	TEXT	required; default datetime('now')
`updated_at`	TEXT	required; default datetime('now')

webhook_events

Column	Type	Notes
`provider`	TEXT	primary key
`event_id`	TEXT	primary key
`received_at`	TEXT	required; default datetime('now')

messages

Column	Type	Notes
`id`	INTEGER	primary key
`order_id`	INTEGER	→ orders (on delete cascade)
`channel`	TEXT	required
`provider`	TEXT	required; default ''
`recipient`	TEXT	required
`event`	TEXT	required; default ''
`subject`	TEXT	required; default ''
`body`	TEXT	required
`status`	TEXT	required; default 'PENDING'
`attempts`	INTEGER	required; default 0
`error`	TEXT	required; default ''
`created_at`	TEXT	required; default datetime('now')
`sent_at`	TEXT	
`payload`	TEXT	required; default ''
`locked_at`	TEXT	

customers

Column	Type	Notes
`id`	INTEGER	primary key
`name`	TEXT	required

Column	Type	Notes
`phone`	TEXT	required
`email`	TEXT	
`password_hash`	TEXT	required
`password_set`	INTEGER	required; default 1. 0 for an account made with Google, Facebook or Apple whose password is a random one nobody knows; set to 1 when the customer chooses a password.
`status`	TEXT	required; default 'ACTIVE'
`notify_sms`	INTEGER	required; default 1
`notify_email`	INTEGER	required; default 1
`notify_whatsapp`	INTEGER	required; default 0
`default_zone_id`	INTEGER	
`created_at`	TEXT	required; default datetime('now')
`last_login_at`	TEXT	
`updates_seen_at`	TEXT	

customer_sessions

Column	Type	Notes
`token_hash`	TEXT	primary key
`customer_id`	INTEGER	required; → customers (on delete cascade)
`expires_at`	TEXT	required
`created_at`	TEXT	required; default datetime('now')
`user_agent`	TEXT	required; default ''

customer_identities

Column	Type	Notes
`id`	INTEGER	primary key
`customer_id`	INTEGER	required; → customers (on delete cascade)
`provider`	TEXT	required; google, facebook or apple
`subject`	TEXT	required; the provider's own id for the person. Unique with provider.
`email`	TEXT	required; default ''. The email the provider shared.
`created_at`	TEXT	required; default datetime('now')

oauth_states

Column	Type	Notes
`state_hash`	TEXT	primary key (hash of the state sent to the provider)
`provider`	TEXT	required
`binding_hash`	TEXT	required (hash of the cookie value that ties the attempt to one browser)
`nonce`	TEXT	required
`verifier`	TEXT	required (PKCE verifier, Google)
`next_path`	TEXT	required; default '/account'
`link_customer_id`	INTEGER	set when a signed-in customer is connecting a provider
`expires_at`	TEXT	required

social_signups

Column	Type	Notes
`token_hash`	TEXT	primary key (hash of the cookie value)
`provider` , `subject`	TEXT	required
`name` , `email`	TEXT	required; default ''
`email_verified`	INTEGER	required; default 0
`next_path`	TEXT	required; default '/account'
`expires_at`	TEXT	required

customer_addresses

Column	Type	Notes
`id`	INTEGER	primary key
`customer_id`	INTEGER	required; → customers (on delete cascade)
`label`	TEXT	required
`recipient`	TEXT	required
`phone`	TEXT	required
`zone_id`	INTEGER	
`address`	TEXT	required
`landmark`	TEXT	required; default ''
`is_default`	INTEGER	required; default 0

password_resets

Column	Type	Notes
`token_hash`	TEXT	primary key
`customer_id`	INTEGER	required; → customers (on delete cascade)

Column	Type	Notes
`expires_at`	TEXT	required
`used_at`	TEXT	

wishlist_items

Column	Type	Notes
`customer_id`	INTEGER	primary key; → customers (on delete cascade)
`product_id`	INTEGER	primary key; → products (on delete cascade)
`created_at`	TEXT	required; default datetime('now')

reviews

Column	Type	Notes
`id`	INTEGER	primary key
`product_id`	INTEGER	required; → products (on delete cascade)
`customer_id`	INTEGER	→ customers (on delete set null)
`author`	TEXT	required
`rating`	INTEGER	required
`title`	TEXT	required; default ''
`body`	TEXT	required; default ''
`status`	TEXT	required; default 'PUBLISHED'
`created_at`	TEXT	required; default datetime('now')

admin_users

Column	Type	Notes
`id`	INTEGER	primary key
`email`	TEXT	required
`name`	TEXT	required
`password_hash`	TEXT	required
`role`	TEXT	required; default 'custom'
`permissions`	TEXT	required; default '[]'
`status`	TEXT	required; default 'ACTIVE'
`must_change_password`	INTEGER	required; default 1
`session_version`	INTEGER	required; default 1
`created_by`	TEXT	required; default ''
`created_at`	TEXT	required; default datetime('now')
`last_login_at`	TEXT	

audit_log

Column	Type	Notes
`id`	INTEGER	primary key
`at`	TEXT	required; default datetime('now')
`actor`	TEXT	required
`action`	TEXT	required
`target`	TEXT	required; default ''
`detail`	TEXT	required; default ''

fx_rate_history

Column	Type	Notes
`id`	INTEGER	primary key
`rate`	REAL	required
`markup_pct`	REAL	required
`note`	TEXT	required; default ''
`changed_at`	TEXT	required; default datetime('now')

order_costs

Column	Type	Notes
`order_id`	INTEGER	primary key; → orders (on delete cascade)
`retailer_gbp_minor`	INTEGER	required; default 0
`uk_delivery_gbp_minor`	INTEGER	required; default 0
`purchase_rate`	REAL	required; default 0
`freight_ghs_minor`	INTEGER	required; default 0
`local_delivery_ghs_minor`	INTEGER	required; default 0
`payment_fees_ghs_minor`	INTEGER	required; default 0
`other_ghs_minor`	INTEGER	required; default 0
`note`	TEXT	required; default ''
`updated_at`	TEXT	required; default datetime('now')

order_tracking

Column	Type	Notes
`id`	INTEGER	primary key
`order_id`	INTEGER	required; → orders (on delete cascade)
`stage`	TEXT	required
`carrier`	TEXT	required; default ''

Column	Type	Notes
`reference`	TEXT	required; default ''
`url`	TEXT	required; default ''
`note`	TEXT	required; default ''
`created_at`	TEXT	required; default datetime('now')

catalog_sources

Column	Type	Notes
`id`	INTEGER	primary key
`shop_id`	INTEGER	required; → shops (on delete cascade)
`name`	TEXT	required
`kind`	TEXT	required
`url`	TEXT	required; default ''
`field_map`	TEXT	required; default '{}'
`terms_url`	TEXT	required; default ''
`terms_note`	TEXT	required; default ''
`terms_confirmed_at`	TEXT	
`enabled`	INTEGER	required; default 0
`auto_publish_new`	INTEGER	required; default 1
`auto_apply_updates`	INTEGER	required; default 1
`max_price_change_pct`	INTEGER	required; default 40
`max_items`	INTEGER	required; default 50
`delay_ms`	INTEGER	required; default 3000
`interval_hours`	INTEGER	required; default 24
`stale_days`	INTEGER	required; default 14
`default_category`	TEXT	required; default ''
`default_weight_grams`	INTEGER	required; default 500
`last_run_at`	TEXT	
`last_status`	TEXT	required; default ''
`last_message`	TEXT	required; default ''
`paused_until`	TEXT	
`running_since`	TEXT	
`created_at`	TEXT	required; default datetime('now')

import_items

Column	Type	Notes
`id`	INTEGER	primary key
`source_id`	INTEGER	required; → catalog_sources (on delete cascade)
`external_id`	TEXT	required
`product_url`	TEXT	required; default ''
`name`	TEXT	required
`brand`	TEXT	required; default ''
`category`	TEXT	required; default ''
`description`	TEXT	required; default ''
`price_minor`	INTEGER	required
`compare_at_minor`	INTEGER	
`image_url`	TEXT	required; default ''
`in_stock`	INTEGER	required; default 1
`weight_grams`	INTEGER	
`options`	TEXT	required; default '[]'
`fingerprint`	TEXT	required; default ''
`status`	TEXT	required; default 'PENDING'
`hold_reason`	TEXT	required; default ''
`product_id`	INTEGER	→ products (on delete set null)
`first_seen_at`	TEXT	required; default datetime('now')
`last_seen_at`	TEXT	required; default datetime('now')

import_runs

Column	Type	Notes
`id`	INTEGER	primary key
`source_id`	INTEGER	required; → catalog_sources (on delete cascade)
`started_at`	TEXT	required; default datetime('now')
`finished_at`	TEXT	
`status`	TEXT	required; default 'RUNNING'
`fetched`	INTEGER	required; default 0
`created`	INTEGER	required; default 0
`updated`	INTEGER	required; default 0
`held`	INTEGER	required; default 0
`skipped`	INTEGER	required; default 0
`removed`	INTEGER	required; default 0

Column	Type	Notes
`message`	TEXT	required; default ''

8. Security and privacy

What protects the system, what personal data it holds, and what is not covered. This describes the code as built; it is not a legal opinion.

8.1 Who can sign in, and how

Who	Credential	Protection
Super admin	`ADMIN_PASSWORD` (a server setting) at `/admin/login?developer=1`	Not stored in the database. Compared in constant time. Cannot be edited or switched off from the admin.
Staff	Email and password at `/admin/login`	Passwords hashed with **scrypt** (N=2^{15}, r=8, p=3, random salt). Minimum **10 characters**, common passwords refused. New accounts must choose their own password before anything else opens.
Customers	Phone (or email) and password, or Google, Facebook or Apple	Passwords: same scrypt hashing, minimum 8 characters, common passwords and a password containing the phone or email refused. Provider sign-in: see below.

Sessions

- **Admin:** a signed cookie (`admin_session`), HMAC-SHA256 with `ADMIN_SECRET` , `HttpOnly` , `SameSite=Lax` , `Secure` in production, limited to `/admin` , valid **8 hours**. A **staff** session also carries the account's **session version**, which is re-checked against the database on **every request**. Switching an account off, changing its rights or resetting its password raises the version, so open sessions end **at once**.
- **Customers:** a random token in a `HttpOnly` , `SameSite=Lax` , `Secure` , `__Host-` prefixed cookie, valid **30 days**; only a **hash** of the token is stored. Customers can sign out other devices.
- **Password resets:** a random one-time token valid **60 minutes**, stored hashed; a successful reset signs the customer in and ends other sessions.

Sign in with Google, Facebook or Apple

- The standard authorisation-code flow over HTTPS. Each attempt carries a random **state**, a **nonce** and (Google) a **PKCE** challenge, lasts **10 minutes**, can be used **once**, and is tied to the browser that began it by a `HttpOnly` , `Secure` , `__Host-` cookie (`SameSite=None` in production, because Apple returns the person with a cross-site form post). Only hashes of the state and the cookie value are stored.
- Google and Apple identify the person by an **ID token** received straight from the provider's token endpoint over TLS; its issuer, audience (our app), expiry and nonce are checked. Facebook's access token is exchanged for the profile with an app-secret proof.
- **Account matching:** by the provider's own id, or by an email address the provider says it has **verified** (Google, Apple). Facebook does not say, so its email never connects to an existing account; a phone number or name never does either. Such a person is told to sign in another way and connect the provider from **Account → Security**.
- Switched-off accounts cannot sign in this way. Provider secrets (and the Apple private key) are saved **encrypted**, like other integration keys. Provider sign-in is **only offered when `APP_URL` is an https address**.
- A first-time sign-in must add a phone number before the account exists. The account gets a random password nobody knows; the customer can choose one through "Forgot your password".
- Start and return addresses are rate-limited per connection.

Throttles (kept in memory, per server)

What	Limit
Admin sign-in	5 failures per place, and 8 per email, per 15 minutes
Customer sign-in	15 per place and 6 per account per 15 minutes
Customer sign-up	10 per place per hour
Password reset requests	6 per place and 3 per account per hour
Order lookup by number	8 failures per 15 minutes
Orders placed	10 per place per hour
Link requests	6 per place per hour
Link preview	12 per place per 10 minutes
Search suggestions	120 per place per minute
Payment return checks	20 per place per 10 minutes

Unknown emails take the same time to reject as wrong passwords, so account existence is not leaked by timing.

8.2 Access control (roles and rights)

- The **super admin** can do everything and is the **only** one who can manage staff accounts. That right is not in the list of rights and can never be granted.
- Each **staff** account holds a list of **rights** (see the table in the Admin guide). The roles are presets that tick rights; the super admin can tick exactly what a person needs.
- **Every admin page and every admin action checks a specific right on the server**, not just in the menu. Typing the address of a page you cannot use shows **No access**. The orders download is checked too.
- Staff cannot grant themselves more access. Changing or removing access takes effect immediately (session version).
- A test fails the build if an admin page or action has no access check. See Testing.
- The **activity log** records sign-ins and every administrative action with the person's name and email.

8.3 Money and payments

- **Card data never touches this server.** Customers pay on the gateway's own hosted page.
- Prices are **always recalculated on the server** from live settings when an order is created; the browser's figures are never trusted.
- A payment is marked paid only after the **amount and currency** match what was asked for. A mismatch is flagged on the dashboard ("Payments with the wrong amount") and does not mark the order paid.
- **Webhooks:** the signature is verified on the raw body before anything is read; for gateways whose webhook is only authenticated by a shared secret, the payment is **re-checked with the gateway** first; each event is applied **once**; bodies over 1 MB are refused; failures return 500 so the gateway retries.
- Only a confirmed payment moves an order to *Payment received*; staff cannot mark an order paid.
- Link-order quotes use a long random private token, expire, and cannot create two orders.

8.4 Secrets

- Provider keys and catalogue feed addresses saved in the admin are encrypted with **AES-256-GCM** using `SETTINGS_ENCRYPTION_KEY` (or `ADMIN_SECRET` if that is not set).
- Keys are shown **masked** in the admin and are never written to logs or the activity log.
- An environment setting always overrides a saved key.
- The public address (`APP_URL`) is never taken from request headers, so a forged `Host` header cannot redirect customers or payments.

8.5 Other protections

Area	Protection
Network fetches by the importer	Only http/https on ports 80 and 443, no credentials in addresses, no private or internal addresses (checked again when the name is resolved), at most three redirects (each re-checked), size and time limits, obeys `robots.txt` , stops on refusals. Prevents the server being used to reach internal systems.
Uploads	Photos and logos are accepted by **file content** (JPEG, PNG, WebP, GIF), not by name; up to 4 MB; stored under random names; served with a fixed type, `nosniff` and a `sandbox` content policy so an upload can never run as a page; path tricks are blocked.
Cookies and sessions	`HttpOnly` , `SameSite=Lax` , `Secure` in production.
Response headers	`X-Frame-Options: DENY` , `X-Content-Type-Options: nosniff` , `Referrer-Policy: strict-origin-when-cross-origin` , a `Permissions-Policy` that turns off camera, microphone and location, and `Strict-Transport-Security` . The `X-Powered-By` header is removed.
Cron endpoints	Answer 404 until `CRON_SECRET` is set, then need `Authorization: Bearer …` .
Admin and search engines	`/admin` , `/account` , `/cart` , `/checkout` , `/pay/` , `/order/` , `/quote/` and similar are disallowed in `robots.txt` .
SQL	All queries use bound parameters.
Link previews	Rate limited; never reach private addresses; obey `robots.txt` .

8.6 Personal data held

Data	Where	Notes
Customer name, phone, optional email, password hash, notification choices, default area	`customers`	Password is hashed.
Saved delivery addresses	`customer_addresses`	Up to 10 per customer.
Sessions	`customer_sessions`	Token hash and a shortened browser description.
Orders: name, phone, email, delivery address, landmark, notes, items, amounts	`orders` , `order_items`	Kept as the business record of a sale.
Link requests: link, details, name, phone, email	`link_requests`	
Messages: recipient, text	`messages`	Recipients are partly hidden in the admin.
Reviews	`reviews`	Shown as first name and last initial.

Data	Where	Notes
Staff: name, email, password hash, rights, last sign-in	`admin_users`	
Activity log	`audit_log`	Staff names and emails as actors.

What deleting an account does: a customer can delete their account (Security page). The account, addresses, wishlist, sessions and reset links are removed; reviews are kept but shown as "Former customer"; link requests are detached from the account. **Orders are kept** (without the link to an account) because they are the record of a sale and contain the delivery details used. Decide your own retention period for old orders and requests; the system does not delete them automatically.

Where data goes: customer contact details are sent to the message providers you enable (to deliver messages), and name, phone and email go to the payment gateway during payment. Photos are shown from their original sources when imported.

8.7 Not covered (be aware)

- **Phone or email verification** at sign-up.
- **Two-step sign-in** for staff or the super admin.
- A **shared rate-limit store**: throttles are per server process and reset on restart (fine for one instance).
- A **Content-Security-Policy** for the main pages (uploads have a strict one).
- **Automatic data deletion** or export for privacy requests; do these by hand.
- **Legal and tax matters**: consumer terms, refunds policy, VAT, import duty, payment licensing and data-protection registration (for example with Ghana's Data Protection Commission, and UK/EU rules if you serve those users). Get professional advice.

8.8 If something goes wrong

Event	Do this
A staff account may be compromised	Super admin: switch it off (signs it out at once), reset its password, check the **Activity log**.
A provider key leaked	Rotate it at the provider, update it in Integrations, press **Test**.
`ADMIN_SECRET` leaked	Change it and redeploy (signs everyone out of the admin). Set a separate `SETTINGS_ENCRYPTION_KEY` first so saved keys stay readable.
The super admin password leaked	Set a new `ADMIN_PASSWORD` and redeploy.
Suspicious payments	Check the gateway dashboard and the "wrong amount" list; do not ship until reconciled.
Customer data exposed	Follow your legal duties to notify; rotate secrets; review the activity log.

9. Integrations

Every outside service the shop talks to, how to set it up, and what the system does with it. Keys are entered in **Admin → Integrations** (needs the *Manage integrations and keys* right) or as environment variables (an environment value always wins). Saved keys are **encrypted** and shown only **masked**.

Each provider card shows: its readiness, the fields to fill in, the **webhook address** to register (for payment gateways), step-by-step instructions, a link to the provider's own documentation, a **switch** and a **Test** button.

The **Readiness** section at the top lists what is still missing, for example "no payment gateway configured".

How the page works

- **Payment gateways:** you can switch on more than one. Customers choose between those that are on and fully configured.
- **Messaging channels** (SMS, WhatsApp, email): for each channel, **choose which provider is used** (where two are available) and switch it on. A channel only sends if its chosen provider is on and complete.
- **Catalogue APIs:** eBay, Diffbot.
- **Customer sign-in:** Google, Facebook, Apple.
- **Exchange rate feed:** choose a provider and a policy (below).
- **Which updates are sent:** for each order status, tick the channels used. A customer's own choices at checkout apply as well (a customer who turned off SMS gets none).

Payments

Customers cannot pay until at least one gateway is on. Card data never reaches this server.

Gateway	Customer pays in	Good for	Webhook address
Paystack	Cedis (GHS)	Ghana cards and Mobile Money (MTN, Telecel, AirtelTigo)	`{APP_URL}/api/webhooks/paystack`
Flutterwave	Cedis (GHS)	Mobile Money, cards and bank payment	`{APP_URL}/api/webhooks/flutterwave`
Stripe	GBP by default, or GHS if your Stripe account can charge cedis (setting "Charge customers in")	International and UK cards	`{APP_URL}/api/webhooks/stripe`

Fields:

- **Stripe:** secret key, **webhook signing secret**; webhook events `checkout.session.completed`, `checkout.session.async_payment_succeeded`, `checkout.session.async_payment_failed`, `checkout.session.expired`.
- **Paystack:** secret key.
- **Flutterwave:** secret key and a **secret hash** (a long random value you invent, entered here and in the Flutterwave dashboard's webhook settings).

How payments are confirmed (so you know what to expect):

1. The customer is sent to the gateway's hosted page. The site records the attempt.

2. When the gateway reports success (by **webhook** and/or when the customer returns to the site), the site checks the **signature**, re-asks the gateway where needed, and compares **amount and currency** with the order.
3. Only if they match is the order marked **Payment received**. A repeated event is ignored. A mismatch appears on the dashboard under "Payments with the wrong amount" and the order is **not** marked paid.

Use the demo payment page only for testing (`ALLOW_DEMO_PAYMENTS`); never in production.

Messages: SMS, WhatsApp, email

Messages go through an **outbox**: an event (payment received, status change, a quote link) queues messages; they are sent right away and **retried up to three times** if the provider fails. After three failures a message waits in **Messages** for you to retry.

Channel	Providers	What you need
SMS	**Arkesel** (Ghana, branded sender name), **Twilio**	Arkesel: API key and a **registered, approved sender name** (up to 11 letters/numbers). Twilio: account SID, auth token, sender number or Messaging Service.
WhatsApp	**Meta WhatsApp Cloud API**, **Twilio**	An approved **utility message template** with three variables (1 customer name, 2 order number, 3 the update), because customers may not have messaged you in the last 24 hours. Meta: permanent access token, phone number ID, template name and language. Twilio: WhatsApp sender, optional template SID.
Email	**Resend**, **Postmark**	A verified sending domain or sender address and the provider's key.

Defaults for **which updates go on which channel** (editable under *Which updates are sent*):

- Payment received, Received at our UK address, On its way to Ghana, Out for delivery, Delivered, Cancelled, Refunded: **SMS, WhatsApp and email**.
- Buying, Bought, Clearing customs: **email only**.
- Awaiting payment: **none**.

Other messages: **password reset** links (email first, then SMS) and **link-order quotes** (email first, then SMS).

Phone numbers are normalised to international format (Ghana numbers starting `0` become `+233…`).

Exchange rate feed (optional)

Your own rate (set on **Pricing**) always decides prices. A feed can only suggest or apply a change **inside limits you set**.

Provider	Key
ExchangeRate-API	API key (the key is part of the request address, so it is never logged)
Open Exchange Rates	App ID (sent in a header)

Policy (Integrations → Exchange rate feed):

- **Manual**: the feed is ignored.
- **Suggest** (default): the Dashboard and Integrations show the market rate and the gap; you apply it yourself. The dashboard warns when the gap passes the **alert** percentage (default 3%).

- **Automatic**: the rate moves to the market rate **only if the move is within the limit** (default 5%); bigger moves wait for you. Your **markup** always applies on top.

Every change is recorded in the rate history and the activity log. To update on a schedule, call `GET /api/cron/fx` hourly or daily with the cron secret.

eBay (catalogue API)

Needs free keys from developer.ebay.com: **App ID** and **Cert ID**, and an **environment** (Production for real listings, Sandbox for test data). Press **Test connection**. Then create a **Catalogue source** of type eBay with your searches. eBay's API licence sets how listing data and photos may be shown and requires a link back to the listing. Read it and check your product pages meet it.

Marketplace Account Deletion. eBay will not enable Production keys until you either subscribe to its account-deletion notifications or attest that you do not keep eBay member data. The importer keeps only public listing facts (title, price, photo, link, category, condition) and deliberately does **not** store the seller's username, so the exemption is accurate: in the developer portal open the keyset's **Alerts & Notifications**, choose **Marketplace Account Deletion**, switch on **Exempted**, and pick **I do not persist eBay data**. If you ever change the importer to keep member data (usernames, user IDs), you must stop using the exemption and implement the notifications.

Diffbot (Product API, paid)

Needs a **token** from your Diffbot dashboard (there is a free trial; after that each product read uses credits). Save it, switch Diffbot on and press **Test connection**, which reads one example page and so uses one credit. Then create a **Catalogue source** of type Diffbot and list the product page addresses, one per line.

- Diffbot downloads the pages for you and returns the name, price, stock, brand and photo. Only prices clearly in pounds are used.
- The shop's own terms still apply, because Diffbot reads the page on your behalf. Only list pages you are allowed to use. The permission box on the source is how you confirm that.
- The site checks each shop's `robots.txt` before asking Diffbot, and sends nothing for a page the shop disallows or when the shop refuses the `robots.txt` request.
- A bad token, no credits left or a rate limit stops the run with a clear message. A page Diffbot cannot read is skipped and counted.
- Cost control: each run reads up to *Products read per run* pages, so credits used per day are roughly that number times the runs per day. A list is never treated as complete, so removing an address does not hide its product at once; it is hidden by the stale rule.

Customer sign-in: Google, Facebook and Apple

These let customers press **Continue with Google / Facebook / Apple** on the sign-in and sign-up pages. Each one needs an app that **you** register with that company; it costs nothing except Apple, which needs a paid Apple Developer Program membership. Set `APP_URL` to your live **https** address first: without it the buttons stay hidden. Under **Admin → Integrations → Customer sign-in** each card shows the **redirect address** to register with the provider. There is nothing to test with a button here: open the sign-in page in a private window and try it. A button shows only while its provider is complete and switched on.

Google: in Google Cloud Console, create an OAuth client of type *Web application*. Add the redirect address (`…/api/auth/google/callback`) as an *Authorised redirect URI*. Paste the **Client ID** and **Client secret**. Publish the consent screen so people outside your own account can use it.

Facebook: at developers.facebook.com create an app and add *Facebook Login*. Add `…/api/auth/facebook/callback` as a *Valid OAuth Redirect URI* and put your domain under *App settings → Basic*. Paste the **App ID** and **App secret**. Switch the app to **Live** or only people with a role on it can sign in. Facebook does not always share an email address.

Apple: in your Apple Developer account create an App ID with *Sign in with Apple*, a **Services ID** for the web with *Sign in with Apple* configured (your domain and the Return URL `…/api/auth/apple/callback`), and a **key** with *Sign in with Apple* (download the `.p8` once). Paste the **Services ID**, **Team ID**, **Key ID** and the `.p8` contents. Apple shares a person's name only the first time, and many people use *Hide My Email*, so the email is an Apple relay address; messages still reach them.

Facebook, Google and Apple will ask for public policy addresses before an app can go live. Use `https://YOUR-DOMAIN/privacy` (privacy policy), `https://YOUR-DOMAIN/terms` (terms of service) and `https://YOUR-DOMAIN/data-deletion` (Facebook's *User data deletion* instructions URL). Fill in the contact email and business name under **Pricing → Site** first.

What customers see: a first-time person is asked for a phone number (we arrange deliveries by phone) and the account is created. A person whose Google or Apple email already has an account is signed in to it. A Facebook email is never matched to an existing account (Facebook does not say it is verified), so that person is asked to sign in another way and connect Facebook from **Account → Security**. Signed-in customers can connect more providers there.

Webhooks and scheduled jobs: addresses

What	Address	Protected by
Stripe / Paystack / Flutterwave webhooks	`POST {APP_URL}/api/webhooks/stripe`, `/paystack`, `/flutterwave`	The gateway's signature
Retry queued messages	`GET {APP_URL}/api/cron/messages`	`Authorization: Bearer $CRON_SECRET`
Sync the market exchange rate	`GET {APP_URL}/api/cron/fx`	same
Run due catalogue sources	`GET {APP_URL}/api/cron/ingest` (only needed if `INGEST_AUTORUN=false`)	same
Health check	`GET {APP_URL}/api/health`	none (returns only `{"ok":true}` or 503)

Adding a new provider (for developers)

Providers are described in `src/lib/integrations.ts` (fields, environment names, steps, webhook). A payment gateway is an adapter in `src/lib/payments/` implementing `PaymentGateway` (`createCheckout`, `check`, `parseWebhook`); message senders are in `src/lib/notify/senders.ts` and chosen in `deliver()` in `outbox.ts`. Add tests alongside (see `payments.test.ts`, `outbox.test.ts`).

10. Testing

The system has two layers of automated checks, plus manual checks you should do on the live site. At the time of writing: **321 unit and database tests in 41 files**, and **40 end-to-end browser steps**.

Commands

Command	What it does
`npm test`	Runs all unit and database tests once (Vitest). Takes about 15 seconds.
`npm run test:watch`	Re-runs tests as you edit.
`npm run lint`	Code style and common mistakes (ESLint).
`npm run typecheck`	TypeScript type check.
`npm run build`	Production build (also type-checks pages).
`npm run e2e`	Builds the shop, starts it on a throwaway database, drives a real browser through everything, and stops it. See `e2e/README.md`.

Before every release run: `npm run lint && npm run typecheck && npm test && npm run e2e`.

Layer 1: unit and database tests

Tests live next to the code they check (`src/lib/*.test.ts`, `src/lib/ingest/*.test.ts`, `src/lib/notify/*.test.ts`, `src/lib/payments/*.test.ts`). Database tests use a **fresh in-memory database** with the real schema (`openForTest()`), so they exercise the real SQL without touching any real data. They run in isolation and can run in any order.

Area	What is checked
Pricing	Service charge modes, rate cards and brackets, rounding, exchange rate and markup, chargeable weight, full order totals.
Orders	Quote and creation, minimum order, payment confirmed once, tracking by number plus contact, legal status steps only, refunds.
Payments	Gateway adapters, webhook signatures, amount and currency matching, events applied once.
Messages	Wording for every status, per-status rules, customer preferences, retries, phone and email handling.
Customers	Registration, sign-in, sessions, password reset, addresses, wishlist, account deletion.
Admin and staff	Staff accounts, password rules, session cut-off when switched off or changed, rights and role presets, session tokens (tamper and expiry), the **guard test** below.
Catalogue importer	Network safety (private addresses, redirects), `robots.txt` rules, price and feed parsing, feed field mapping, publishing, price-move guard, removal rules, eBay, Shopify, file import, source-kind migrations.
Link orders	Quotes, expiry, one order per quote, automatic-quote rules, margin arithmetic, item types, one-click submit.
Catalogue and shops	Deleting shops and delivery areas, shop logos, department icons, themes, seeding and cleanup, search and filters.

Area	What is checked
Security helpers	Encryption of saved keys, uploads (file types, size), throttling, cron protection.

The guard test (`src/lib/admin-guards.test.ts`)

It reads the source and **fails if any admin page or admin action lacks an access check**, if staff management is not super-admin-only, or if the orders download does not check its right. When you add a new admin page or action, it must call `requirePermission("…")` (or `requireSuper()` / `requireAdmin()` where appropriate) or the test fails.

Layer 2: end-to-end browser run

`e2e/e2e.mjs` uses a real headless Chromium and checks, in order: protected pages and unsigned webhooks refused; search; adding to cart; sign-up; checkout and payment; account and tracking; admin sign-in; every admin page loads; order status changes; exchange rate and pricing changes; catalogue sources (permission, internal addresses refused, create and remove, eBay and Shopify forms, file import); shops (logo upload, delete) and delivery areas (add, delete); themes; photo uploads (including refusing a fake image); the Add-by-link button and the home-page link box; link requests, quotes, payment and automatic quotes; one-click found-link flow; **staff accounts** (create, forced password change, limited menu, "No access", no change buttons, no export, switch off signs out); phone layouts for the admin and the shop; wishlist.

`run.sh` also starts a second throwaway shop whose `APP_URL` is an https address, because the Google, Facebook and Apple buttons only show on https; the "social sign-in" step uses it to check the button appears once Google is set up, the redirect to Google carries state, nonce and PKCE, and a stray return is refused. Nothing is ever sent to that address.

It fails if the browser console shows errors.

What is deliberately *not* tested automatically

- **Real shops and real payments.** The test environment cannot reach retailer sites or payment gateways. Reading a real page, a real gateway checkout and real SMS/WhatsApp/email delivery must be checked by hand on the live site.
- **Real phones.** Phone layouts are checked in an emulated phone in Chromium, not on iPhone Safari or specific Android browsers.
- **Load.** There is no load test; the system is built for one small shop.

Manual checklist before announcing the shop

1. `/api/health` answers `{"ok":true}`.
2. Sign in as the super admin and as a test staff account; confirm the staff menu is limited.
3. Place a **small real order** with a **real payment** through each gateway you switched on; confirm the order becomes *Payment received* and the customer message arrives.
4. Move that order through every status and check the customer sees each step and message.
5. Cancel and refund a test order and mark it **Refunded**.
6. Paste a real product link on the home page; confirm the price shown matches the shop; send a link request you must quote by hand and complete it.

7. Run a catalogue source and check items, photos and prices against the shop.
8. Open the site and the admin on a real phone (iPhone and Android if you can).
9. Make a backup and **restore it** on a copy.

Adding tests

- Logic goes in `src/lib` with a `*.test.ts` beside it. Use `openForTest()` for anything that touches the database.
- Anything that handles money, access or the network needs tests before it is changed.
- A bug fix should start with a test that fails because of the bug.

11. FAQ and troubleshooting

For the operator

Customers say they cannot pay

Check **Admin → Integrations → Readiness**: a payment gateway must be configured **and switched on**. Check the gateway's own dashboard for errors, and that `APP_URL` is your live `https` address with the gateway webhook registered. On the Dashboard look for "Payments with the wrong amount".

A customer paid but the order still says "Awaiting payment"

The gateway's confirmation has not reached the site (or the amount did not match). Check the order's **Payments** panel, the gateway dashboard, and the webhook delivery log at the gateway. Staff cannot mark an order paid by hand. If the gateway shows the payment as successful, **resend the webhook** from the gateway's dashboard (most have a resend button); the site applies each event once, so resending is safe. If the gateway shows no payment, the customer has not paid.

How do I refund?

Cancel the order (allowed up to *Buying from the UK shop*), refund the customer **in your gateway's dashboard**, then open the order and click **Mark as: Refunded**. The site does not move money itself.

An order was cancelled after payment and shows on the dashboard

That is the "Paid orders that need a refund" reminder. Refund and mark it refunded.

I changed the exchange rate: do old orders change?

No. Each order stores the rate, markup and every amount at the time it was placed.

Prices on the site look wrong

Prices are the **UK price × effective rate** (rate plus markup) plus service charge, shipping and delivery. Open **Admin → Pricing → What customers pay now** to see a worked example, and check **Shipping** (rate card) and **Delivery areas**. Imported prices come from the source; check the source and its last run.

A product has no photo

A grey icon shows when an item has no photo or the photo link is dead. Upload a photo in **Items**, or check the feed's image column. Imported images are linked from the source.

Products disappeared

Imported items are hidden when their source stops refreshing them (the "hide items not refreshed for N days" setting), when they vanish from a complete feed, or when a source was removed with its products. Check **Catalogue sources** → the source → recent runs.

A catalogue source says "Blocked by the shop"

The shop refuses automated reading (HTTP 403 and similar). The importer will **not** try to get around this. Use an affiliate feed, link orders, or remove the source. See Catalogue sources.

The eBay source says to add keys

Enter the App ID and Cert ID in **Integrations → Catalogue APIs**, switch eBay on, and press **Test connection**.

The Diffbot source reads nothing, or says "the shop's robots.txt does not allow it"

The shop's `robots.txt` disallows that page (or the shop refused even the request for `robots.txt`), so nothing was sent to Diffbot. That is deliberate and is not a Diffbot fault. Use the shop's affiliate feed instead. If instead the run stops with "did not accept the token" or "credits have run out", fix the token under Integrations or add credits at Diffbot.

"We could not complete that sign-in" after Google, Facebook or Apple

The person reached the provider and came back, so the shop's own check failed. Open the server log (Render: your service, **Logs**) and search for `[social:`. The line names the cause, for example:

- `token exchange: HTTP 400, Error validating client secret` (Facebook) or `invalid_client` (Google): the App secret or Client secret saved under **Integrations → Customer sign-in** is wrong or was regenerated. Paste it again.
- `Error validating application` or `Invalid app id`: the App ID is wrong, or the keys are for another app.
- `redirect_uri` problems (`Redirect URI` / `redirect_uri_mismatch`): the address registered with the provider differs from the one on the card. Compare character by character (https, www, no trailing slash) and check `APP_URL`.
- `the provider sent the person back with an error`: the provider refused before returning a code (for example the app is not Live and the person has no role on it).
- `no usable code, or this provider is no longer set up`: the keys were removed or switched off, or `APP_URL` is not https.

The WooCommerce source says "does not look like a WooCommerce shop"

Open `theirshop/wp-json/wc/store/v1/products`. If it is not a block of text starting with `[{"id":`, the shop is not on WooCommerce or has switched its public list off, and it cannot be read. Ask the owner for a product feed instead. If the text shows a currency other than `GBP`, the shop cannot be used.

The Shopify source says "does not look like a Shopify shop"

Open `theirshop/meta.json`. If it is not a short block of text with `"currency":"GBP"`, the shop is not on Shopify or does not price in pounds.

A staff member cannot see a page / button

Their account does not have that right. The super admin can change it under **Staff accounts**. Changing it signs them out; they sign in again to get the new access.

I forgot the super admin password

Set a new `ADMIN_PASSWORD` in the host's settings and redeploy. See Deployment → runbooks.

A staff member forgot theirs

Super admin → **Staff accounts** → the person → **Reset password**.

Messages are not being sent

Admin → Messages shows each failure and why. Typical causes: missing or wrong provider key, an unapproved SMS sender name or WhatsApp template, the channel not switched on, or the status not ticked under **Which updates are sent**. Press **Retry** after fixing.

Link request: the customer says the link does not work

The quote may have **expired** (it is held for the number of days you set), or the customer is not signed in as the same person. Open the request, **Change the quote** (this keeps the same link and gives it a new expiry) and send it again.

The price the system read for a link is wrong

Large shop pages can contain several prices. Switch off **Quote by itself when the price is read from the shop's own web page** under **Link requests → Automatic quotes** so every request waits for your team, and check how many were wrong before turning it back on.

Which devices and browsers are supported?

Built for current versions of the major browsers on desktop, Android and iPhone, and for phone widths from about 320 pixels. It has been tested in Chrome (desktop and an emulated phone); check Safari on a real iPhone yourself.

For customers (quick answers)

Do I need an account?

To **check out**, yes. You can browse and look up an order without one.

Is my card safe?

You pay on the payment partner's own page. We never see or store your card details.

Why is there a service charge?

It covers buying, handling and arranging shipment for you. It is shown as its own line.

Are customs duties included?

No. Any duty or taxes charged by customs in Ghana are not included in the total.

I pasted a link and it said it could not read the page

Some shops do not allow automatic reading. Choose **Request it anyway**: we check the price and send you a link to pay.

Can I cancel?

Before we have bought the item (up to *Buying from the UK shop*), yes: contact us. After that it usually cannot be cancelled.

I forgot my password

Use **Forgot password** on the sign-in page. The link works for 60 minutes.

Error messages you may see

Message	Meaning
"Too many attempts. Try again in 15 minutes."	Sign-in is temporarily blocked after repeated failures. Wait, or ask the super admin to reset the password.
"That email and password do not match an active staff account."	Wrong details, or the account is switched off or deleted.
"No access"	Your account does not have the right for that page.
"The minimum order is £..."	The items total is below the minimum set on **Pricing**.
"This quote has expired..."	A link-order price was held for its set days and the time passed. Ask for a fresh one.
"Online payment is not set up yet."	No gateway is configured, or `APP_URL` is not an `https` address.
"Admin is locked. Set ADMIN_PASSWORD and ADMIN_SECRET..."	Those two settings are missing on the server.

12. Change history

What was built and in what order. Every change below is a commit on the `claude/pensive-meitner-d8kx3v` branch; run `git log` for the exact commits. All work so far happened on one day, 7 October 2026, in a series of steps agreed with the owner.

Foundations

Step	What was added
Platform	The storefront, customer accounts, order flow with tracking, payments (Stripe, Paystack, Flutterwave), messages (SMS, WhatsApp, email), and the admin console with dashboard, orders, customers, shops, items, pricing, shipping, delivery areas, integrations, messages and activity log. Exchange rate set by the admin, with an optional market feed.
Marketplace storefront	Marketplace-style layout (deals, departments, shelves, search with suggestions, filters, recently viewed, wishlist), reviews with moderation.
Catalogue importer	Catalogue sources that fill the shop from feeds and pages, with strict safety rules (robots.txt, polite pacing, stop on refusal, no private addresses), review queue and scheduler.
Look and themes	A distinct Ghana green and gold look with its own fonts and shapes, marquee and animations, and 15 colour themes chosen in the admin.
Rename and deployment	Renamed to **SHOP UK FROM GH**. Deployment package: Docker image, Compose with HTTPS, Fly.io and Render configurations, health check, backups guidance.
Rebrand	Renamed to **YenkoExpress** (domain **yenkoexpress.com**). Existing installs are renamed at start-up; the hosting service name, the encryption salt and the `UKG-` order-number prefix were kept so nothing already stored breaks.
Logo	The YenkoExpress logo (cart icon and stacked wordmark) is on the header, footer, sign-in pages and admin, and is the favicon and phone home-screen icon. Source files and the app icons for Google and Facebook sign-in are in `docs/brand/` ; the web copies are in `public/brand/` .
No discount claims	Removed every percentage-off tag, "was" price, "You save" amount, "Limited-time deal" label, deal countdown, the Today's deals shelf and links, the deals filter and the biggest-discount sort. Sources' was-prices are no longer stored on items, and earlier ones were cleared. The admin no longer has Was price or Deal ends fields.
Add-on listings	Warranty, protection-plan and fee listings (for example a £1 "Worry-Free Purchase") are not products. New ones are held for review, and ones already live that are certainly add-ons are taken off the site, out of carts, and marked rejected when the server starts.

Deployment fixes

- Fixed the Docker build (missing `public` folder) and Render disk permissions (the container now fixes ownership of the data folder, then drops root).
- Render region set to **Frankfurt**.
- `robots.txt` added. (The public footer no longer links to the admin: staff sign in at `/admin/login` .)

Products and photos

- Product photo upload and a visible **Add by link** button.
- Category pictures for items without photos; full-size merchant photos preferred in feeds; a "get real products" guide on the Catalogue sources page.

- Fictitious shops removed. A new database now contains only **eBay UK**.

Catalogue sources

- **eBay** official-API source.
- **Remove source** (keep or remove its products).
- **Shopify** shop source with sizes and colours.
- **Catalogue reader page removed** (`/bot` and its footer link) at the owner's request; the importer's user agent now carries the site's address instead.
- **Privacy policy, terms of service and data-deletion pages**, linked from the footer, with contact email and business name set in Pricing → Site; accounts made with a provider can be deleted without a password.
- **Sign in with Google, Facebook and Apple** for customers, with safe account matching, a phone-number step, and connecting providers from Account → Security.
- **Diffbot** (Product API) source: reads product pages you list, checking `robots.txt` first; paid per product read.
- **WooCommerce** shop source (public Store API product list) with sizes and colours.
- **File import** (CSV or JSON you upload).
- Page titles tidied (shop's own tail removed).

Shops and delivery

- **Delete a shop** (with confirmation; orders keep their record) and **shop logos** (upload or link; shown on the shop list, shop page, product page and cart).
- **Delete delivery areas** (the last active one is protected).
- Fixed a fault where every shop form used the same field ids.
- Fixed a restart fault: starter data is no longer re-created if every shop has been deleted.
- **Department icons**: 40 line icons chosen from the department's name.

Link orders

- Pasting a link became the main way to shop: a large box on the home page, header button, search box that recognises links, a **Paste link** tab on phones.
- **Link orders**: link requests are captured, quoted by staff, paid through normal checkout and become ordinary orders.
- **Automatic quotes**: from the shop page's own price, optionally from the customer's typed price plus a margin, with an automatic limit and item-type weights. Link orders show how the price was found.
- **One-click** from a found link to the customer's price and payment.
- Fixed header pop-up panels closing on Safari-style browsers; fixed a rounding error in the safety margin.

Ordering from Amazon UK

- Amazon UK links are recognised by their ASIN (including short links and shared text) and cleaned; other Amazon stores are refused with a pointer to the UK link. The shop never requests an Amazon page.
- A hand-off link from the search page to Amazon UK's search, an **Order from Amazon UK** page with a one-click bookmarklet, and an installable phone app with a Share-menu entry so a customer can send an

Amazon item straight into a request.

- **Request any item**: a customer can describe what they want with no link; staff get Amazon UK / Google UK searches, attach the link they find and quote.
- Customer-typed price stays a guide; staff verify before buying. No Amazon data feed or paid service is used.

Staff and access

- **Roles and access control**: the developer's password is the **super admin**; staff accounts are created by the super admin with roles or exact rights, forced first-time password change, switch-off with immediate sign-out, and an activity log that names the person. Every admin page and action checks a right, and a test enforces this.

Phones

- **Admin on a phone**: Menu button, no sideways scrolling, lists as cards.
- **Shop on a phone**: account page no longer wider than the screen, 16-pixel form fields (stops iPhone zoom), 44-pixel tap targets, bottom bar clear of the home indicator, browser bar takes the shop's colour.

Branding

- "Powered by Anknovate IT Services" in the footer.

Documentation and testing

- This documentation set, and the end-to-end browser checks added to the repository (`npm run e2e`).

Known gaps (carried forward)

- No phone or email verification at sign-up; no two-step sign-in for staff; no email invitation for new staff.
- Rate limits are per server process (single instance only).
- No Content-Security-Policy on the main pages.
- No automatic data deletion or export for privacy requests.
- Real-shop page reading, real payments and real message delivery cannot be tested automatically and need checking on the live site.
- Legal, tax, customs-duty and payment-licensing questions are not addressed by the software.

Software Requirements Specification (SRS)

System: YenkoExpress **Document:** SRS, **as-built** edition **Version:** 1.0 **Date:** 7 October 2026 **Prepared for:** the business owner (product owner) and the development and operations teams **Structure:** follows ISO/IEC/IEEE 29148 and the IEEE 830 outline

> **How to read this document.** It states what the system *shall* do, and for every requirement it records whether the system **does it today**. Status values: **Implemented**, **Partial** (works with a stated limit) and **Not implemented** (a real gap, listed in Appendix B). Requirements are numbered so they can be traced to tests (section 5). Other documents in this folder explain *how* to use, run and build the system; this one defines *what* it must do.

Contents

1. Introduction
2. Overall description
3. External interface requirements
4. Functional requirements
5. Verification and traceability
6. Non-functional requirements
7. Data requirements
8. Business rules
9. Use cases
10. Constraints, assumptions, dependencies and risks

- Appendix A: Requirement status summary
- Appendix B: Gaps and future work
- Appendix C: Glossary
- Appendix D: Revision history

---

1. Introduction

1.1 Purpose

This SRS specifies the requirements of **YenkoExpress**, a web system that lets shoppers in Ghana buy goods from UK retailers and pay once in Ghana cedis (GHS), while the operator buys the goods in the UK, ships them to Ghana and has a third-party courier deliver them. It is the reference for deciding whether the system is correct and complete, for planning changes, and for acceptance testing.

1.2 Scope

In scope. A public storefront; customer accounts; carts, checkout and online payment; order tracking; notifications by SMS, WhatsApp and email; a catalogue that fills itself from approved sources; "link orders" for items from shops that are not in the catalogue; an administration console with role-based access for the owner's staff; and the operational machinery to run it (scheduler, health check, audit log).

Out of scope. Physical purchasing, UK warehousing, international freight and the Ghana courier (all done by people outside the software); customs clearance; accounting and tax filing; stock holding (the business holds no stock); a native mobile app; returns and dispute handling beyond cancel and refund; any marketplace of independent sellers.

Benefits. Customers see the full cost in pounds and cedis before paying, pay with local methods, and can follow the order. The operator controls prices and costs, can serve almost any UK shop, and works from a phone.

1.3 Definitions

See Appendix C. Key terms: **catalogue source**, **link order**, **quote**, **effective rate**, **rate card**, **super admin**, **staff account**, **right**.

1.4 References

- This documentation set: Overview, Admin guide, Customer guide, Catalogue sources, Deployment and operations, Architecture, Data model, Security and privacy, Integrations, Testing.
- ISO/IEC/IEEE 29148:2018 and IEEE 830-1998 (document structure).
- Provider documentation for Stripe, Paystack, Flutterwave, Arkesel, Twilio, Meta WhatsApp Cloud API, Resend, Postmark, ExchangeRate-API, Open Exchange Rates the eBay Browse API and the Diffbot Product API (linked from the Integrations page).

1.5 Conventions

- **shall** = mandatory; **should** = expected unless there is a stated reason; **may** = optional.
- IDs: `FR-` functional, `NFR-` non-functional, `BR-` business rule, `DR-` data, `IR-` interface, `UC-` use case.
- **Priority** uses MoSCoW: **M**ust, **S**hould, **C**ould.
- Money is always integer pence (GBP) or pesewas (GHS).

1.6 Overview of the document

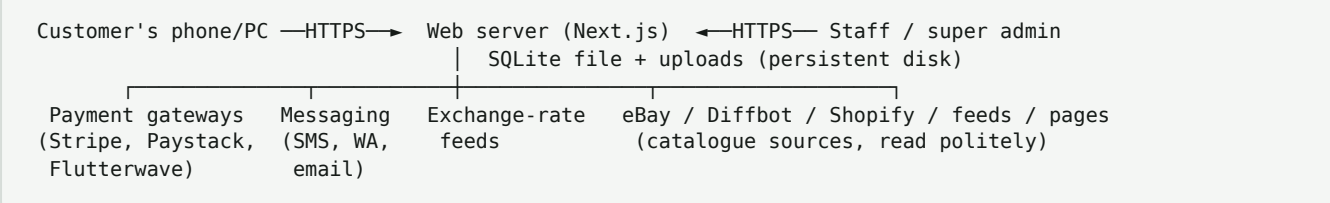
Section 2 describes the product and its users. Section 3 the interfaces. Section 4 the functional requirements, grouped by area. Section 5 how each is verified. Sections 6–8 the quality attributes, data and business rules. Section 9 the main use cases. Section 10 constraints, assumptions and risks.

---

2. Overall description

2.1 Product perspective

A self-contained web application with no dependency on any other system of the business. It depends on **external services** for payments, messaging, exchange rates and catalogue data (section 3.4). It runs as **one server process with a single SQLite database file** on a persistent disk, and is hosted on Render (Frankfurt).



2.2 Product functions (summary)

1. Browse and search a catalogue of UK shops and items with prices in pounds and cedis.
2. Cart, checkout, and online payment in cedis (or pounds where configured).
3. Order lifecycle with tracking, history and notifications.
4. Customer accounts, saved addresses, wishlist, verified-purchase reviews.
5. Link orders: buy an item from any UK shop by pasting its link, with manual or automatic quoting.
6. Catalogue feeding: eBay, Shopify, feeds, sitemap pages, file import, pasted links, with a review queue.
7. Administration: dashboard, orders, customers, requests, catalogue, pricing, shipping, delivery, appearance, integrations, messages, activity log.
8. Staff accounts with roles and fine-grained rights; a super admin that only the developer can use.
9. Operations: scheduler, health check, cron endpoints, encrypted secrets, backups guidance.

2.3 User classes and characteristics

Class	Description	Skills and access
Customer (shopper)	Person in Ghana ordering for themselves. May be a first-time online buyer, usually on a phone with a mobile-data connection.	Needs simple, fast, low-bandwidth pages. Signs in with phone or email.
Visitor	Browses or looks up an order without an account.	No sign-in.
Staff	The owner's employees, each with their own account and only the rights granted (for example customer support, operations, catalogue, finance).	Non-technical. Often on a phone.
Super admin	The developer. Signs in with a server-held password. Can do everything and is the only one who manages staff.	Technical.
Payment gateway / messaging provider	External systems that call the webhook endpoints or are called by the system.	Machine.
Scheduler	The in-process timer and external cron callers.	Machine.

2.4 Operating environment

- **Server:** Node.js 22, Linux container (Docker), one instance, persistent disk, HTTPS terminated by the host.
- **Clients:** current versions of Chrome, Edge, Firefox and Safari on desktop; Chrome and Safari on Android and iPhone; viewport widths from 320 px.
- **Database:** SQLite (single file, write-ahead logging).

2.5 Design and implementation constraints

- **DC-1** One running instance only (single-writer database, in-memory throttles, in-process scheduler).
- **DC-2** Must run on a host with a persistent disk; serverless hosts are not supported.
- **DC-3** Card data shall never be handled by the system; payment uses gateway-hosted pages.
- **DC-4** The importer shall obey `robots.txt` and shall never evade access controls (see FR-CAT-30 to FR-CAT-33).
- **DC-5** Technology: Next.js 16 (App Router), React 19, TypeScript, Tailwind CSS 4, better-sqlite3, zod.
- **DC-6** All money in integer minor units; one pure pricing module used everywhere.

2.6 Assumptions and dependencies

- **A-1** The operator has accounts with at least one payment gateway before taking orders.

- **A-2** Customers have a Ghana mobile number; messages are delivered in English.
- **A-3** The operator has permission, or a lawful basis, to show the catalogue data and images it imports.
- **A-4** UK retailers accept orders delivered to a UK address controlled by the operator.
- **A-5** The operator handles customs duty, VAT and consumer-law obligations outside the software.
- **D-1** Payment, messaging, exchange-rate and eBay services are available and their keys valid.
- **D-2** The host provides HTTPS, a persistent disk and a stable public address (`APP_URL`).

---

3. External interface requirements

3.1 User interfaces

ID	Requirement	Pri	Status	Verified by
IR-UI-01	The storefront shall be usable on phone, tablet and desktop, with no page wider than the screen at widths from 320 px.	M	**Implemented**	E2E "the shop fits a phone"
IR-UI-02	The admin shall be usable on a phone: a Menu button replaces the sidebar, wide lists become labelled cards, no sideways scrolling.	M	**Implemented**	E2E "the admin fits a phone"
IR-UI-03	Form fields shall use text of at least 16 px on phones (to prevent iPhone zoom) and interactive targets shall be at least 44 px tall.	S	**Implemented**	E2E (field size check); inspection
IR-UI-04	Every price shall be shown in **both pounds and cedis** on the product, cart and checkout pages.	M	**Implemented**	Inspection; E2E checkout total
IR-UI-05	The look shall be themeable: the admin chooses one of 15 colour themes that applies to the whole shop.	S	**Implemented**	`themes.test.ts` ; E2E "admin colour theme reaches the storefront"
IR-UI-06	A prominent paste-a-link entry shall be available on the home page, in the header, in the search box and as a phone tab.	S	**Implemented**	E2E "paste-a-link box…"
IR-UI-07	Images that fail to load shall fall back to a category icon (items) or the first letter on the shop colour (shop logos).	S	**Implemented**	Inspection
IR-UI-08	Interface text shall be in plain English suitable for first-time online buyers.	S	**Implemented**	Inspection
IR-UI-09	Screens shall meet WCAG 2.1 AA for colour contrast, labels, keyboard use and focus.	S	**Partial (labels, focus, keyboard and landmarks are implemented; no formal audit)**	Inspection

3.2 Hardware interfaces

None. The system is a web application and uses no special hardware.

3.3 Software interfaces (internal)

- **IR-SW-01** Database: SQLite through `better-sqlite3` , schema in `src/lib/schema.ts` .
- **IR-SW-02** Browser storage: a cart cookie, a customer session cookie, an admin session cookie, a delivery-area cookie, and a local list of recently viewed items. No third-party cookies or trackers.

3.4 Communication and external-service interfaces

ID	Interface	Direction	Requirement	Status
IR-EX-01	**Stripe**	out and webhook in	Create hosted checkout in GBP or GHS; receive signed events (`checkout.session.*`).	**Implemented**
IR-EX-02	**Paystack**	out and webhook in	Create hosted checkout in GHS; verify and receive signed events.	**Implemented**
IR-EX-03	**Flutterwave**	out and webhook in	Create hosted payment in GHS; webhook authenticated by shared hash and re-verified with the gateway.	**Implemented**
IR-EX-04	**Arkesel**, **Twilio**	out	Send SMS (Arkesel: branded sender name).	**Implemented**
IR-EX-05	**Meta WhatsApp Cloud API**, **Twilio**	out	Send approved template messages.	**Implemented**
IR-EX-06	**Resend**, **Postmark**	out	Send transactional email (text and HTML).	**Implemented**
IR-EX-07	**ExchangeRate-API**, **Open Exchange Rates**	out	Fetch the GBP to GHS market rate.	**Implemented**
IR-EX-12	**Diffbot Product API**	out	Read each listed product page by token; only prices in GBP; stop on bad token, no credits or rate limit.	**Implemented**
IR-EX-08	**eBay Browse API**	out	OAuth client-credentials, search UK fixed-price listings in GBP.	**Implemented**
IR-EX-09	**Shop websites and feeds**	out	Fetch feeds, sitemaps, Shopify `/meta.json` and `/products.json` , WooCommerce `/wp-json/wc/store/v1/products` , and product pages, subject to FR-CAT-30 to FR-CAT-33.	**Implemented**
IR-EX-10	**Inbound endpoints**	in	`/api/webhooks/{stripe,paystack,flutterwave}` ; `/api/cron/{messages,fx,ingest}` (bearer secret); `/api/health` .	**Implemented**
IR-EX-13	**Sign in with Google, Facebook and Apple**	out	OpenID Connect and OAuth 2.0 authorisation-code flows over HTTPS; Google and Apple identify the person by an ID token from the token endpoint, Facebook by its Graph profile.	**Implemented**
IR-EX-11	**All outbound calls**	out	Shall use HTTPS where the provider offers it and shall have time and size limits (verified by inspection and `net.test.ts`).	**Implemented**

---

4. Functional requirements

Each table lists the requirement, its priority (M/S/C), its status, and how it is verified. "E2E" means a step of the browser run (`npm run e2e`); file names refer to unit tests in `src/lib` .

4.1 Browsing and search (FR-BRW)

ID	Requirement	Pri	Status	Verified by
FR-BRW-01	The system shall show a home page with featured banners, a paste-a-link entry, departments, new arrivals, best sellers, and recently viewed items.	M	**Implemented**	E2E "home renders deals shelf"
FR-BRW-02	The system shall list shops and show a page per shop with its logo, description, department and items, filterable by item category and sortable.	M	**Implemented**	E2E; `browse.test.ts`

ID	Requirement	Pri	Status	Verified by
FR-BRW-03	The system shall show a page per department (the category a shop is filed under) listing its items and shops, with an icon chosen from the department name.	S	**Implemented**	`department-icons.test.ts`
FR-BRW-04	The system shall offer text search with suggestions of items, shops and departments while typing (at least 2 characters), rate-limited to 120 requests per minute per visitor.	M	**Implemented**	E2E "search suggests products as you type"; `discovery.test.ts`
FR-BRW-05	A pasted web address in the search box shall be recognised and sent to the link flow instead of a text search.	S	**Implemented**	E2E "search box recognises pasted links"
FR-BRW-06	Results shall be filterable by department, shop, item category, price range (typed in cedis), and sortable by featured, price low-to-high, price high-to-low, best rated and newest, 24 per page.	M	**Implemented**	`browse.test.ts`
FR-BRW-07	Invalid or hostile query parameters shall fall back to safe defaults.	M	**Implemented**	`browse.test.ts`
FR-BRW-08	The product page shall show photos (or a fallback), price in pounds and cedis, size/colour options, quantity, the shop, a link to the item on the shop's own website, reviews and rating, related items from the same shop, a wishlist button and a share-on-WhatsApp link.	M	**Implemented**	E2E; inspection
FR-BRW-09	The product page shall show "what it costs to your door" for the chosen delivery area and shipping method.	M	**Implemented**	`pricing.test.ts`; inspection
FR-BRW-10	The visitor shall be able to choose a delivery area that is remembered (cookie; for signed-in customers also saved as their default) and used for "to your door" prices across the shop.	S	**Implemented**	Inspection
FR-BRW-11	Only items marked active, of shops marked shown, shall appear to customers.	M	**Implemented**	`discovery.test.ts` ("hides products from inactive shops")
FR-BRW-12	The system shall make no discount claims: no percentage-off tags, "was" prices, savings amounts, "deal" labels or countdowns. Prices a source reports as a "was" or list price shall not be stored on items or shown.	M	**Implemented**	`discovery.test.ts` ("no discount claims"); `run.test.ts`

4.2 Cart (FR-CRT)

ID	Requirement	Pri	Status	Verified by
FR-CRT-01	A visitor shall be able to add an item, choosing every option (such as size and colour) it requires, without signing in.	M	**Implemented**	E2E "added 2 trainers to the cart"
FR-CRT-02	The same item with the same options shall merge into one line; quantity shall be limited to 1 to 20 per line.	M	**Implemented**	Inspection (`cart.ts`)
FR-CRT-03	The cart shall persist for the visitor through a random token cookie and survive sign-in.	M	**Implemented**	E2E "registered and returned to checkout with the cart intact"
FR-CRT-04	The cart shall show line prices and totals in cedis and pounds, and the progress towards the minimum order.	M	**Implemented**	E2E "cart total"
FR-CRT-05	A visitor shall be able to change quantity, remove a line, and open a cart drawer from any page; a cart count shall show in the header.	M	**Implemented**	E2E
FR-CRT-06	The cart shall price from live data on every view; the browser's figures shall never be trusted.	M	**Implemented**	`pricing.test.ts`; `admin.test.ts` ("prices from live settings")

4.3 Pricing (FR-PRC)

ID	Requirement	Pri	Status	Verified by
FR-PRC-01	The customer total shall equal items + service charge + shipping to Ghana + delivery in Ghana.	M	**Implemented**	`pricing.test.ts`
FR-PRC-02	Item prices shall be converted at the effective rate = admin rate × (1 + markup%).	M	**Implemented**	`pricing.test.ts`
FR-PRC-03	The service charge shall be configurable as a percentage with a minimum, a flat amount, or tiered bands on the item total, and shall be shown on its own line.	M	**Implemented**	`pricing.test.ts` ; `admin-parse.test.ts`
FR-PRC-04	Shipping shall be taken from the chosen method's rate card: weight brackets, a charge per extra kilogram above the largest bracket, and a minimum charge, by chargeable weight (greater of actual and volumetric where known).	M	**Implemented (items carry actual weight only)**	`pricing.test.ts`
FR-PRC-05	Delivery shall be a flat fee per delivery area.	M	**Implemented**	`pricing.test.ts`
FR-PRC-06	The system shall enforce a minimum order value (items only, in pounds) set by the admin; default £10.	M	**Implemented**	`admin.test.ts` ("enforces the minimum order")
FR-PRC-07	One pure function shall price the product page, cart, checkout and the stored order, so the displayed amount equals the charged amount.	M	**Implemented**	`pricing.test.ts` ; E2E "checkout total matches cart"
FR-PRC-08	An order shall store the rate, markup and every amount at the time it is placed; later setting changes shall not alter it.	M	**Implemented**	`admin.test.ts` ; schema inspection
FR-PRC-09	The admin shall see a worked example of what customers pay while editing pricing.	C	**Implemented**	Inspection
FR-PRC-10	Import duty and taxes charged in Ghana shall not be included, and the system shall say so at checkout.	M	**Implemented**	Inspection

4.4 Customer accounts (FR-ACC)

ID	Requirement	Pri	Status	Verified by
FR-ACC-01	A visitor shall be able to register with name, a phone number (stored in international form), an optional email and a password (at least 8 characters; common or personal passwords refused).	M	**Implemented**	`customers.test.ts` ; `password.test.ts` ; E2E
FR-ACC-02	A customer shall sign in with phone or email and password. Passwords shall be hashed with scrypt; an unknown identifier shall take the same time as a wrong password.	M	**Implemented**	`customers.test.ts` ; `password.test.ts`
FR-ACC-03	Sessions shall last 30 days, be stored hashed, and be revocable; a customer shall be able to sign out other devices.	M	**Implemented**	`customer-session.test.ts`
FR-ACC-04	A customer shall be able to reset a forgotten password with a one-time link valid for 60 minutes, sent by email or SMS; using it shall end all other sessions.	M	**Implemented**	`customers.test.ts`
FR-ACC-05	A customer shall be able to edit their profile, change their sign-in details and password, and choose SMS, email and WhatsApp notifications.	M	**Implemented**	`customers.test.ts`

ID	Requirement	Pri	Status	Verified by
FR-ACC-06	A customer shall be able to keep up to 10 saved addresses, one of them default, each with recipient, phone, delivery area, address and landmark.	S	**Implemented**	`customers.test.ts`
FR-ACC-07	A customer shall have a wishlist and see their order history, per-order tracking, an updates feed (with unread marking), live tracking of the latest order, and "buy again".	S	**Implemented**	`account-orders.test.ts`; E2E "wishlist", "order appears in the account"
FR-ACC-08	A customer shall be able to delete their account. Reviews shall remain shown as "Former customer", link requests shall be detached, orders shall be kept without an account link.	S	**Implemented**	`customers.test.ts`
FR-ACC-09	Staff shall be able to disable and re-enable a customer account (signing the customer out everywhere) and create a one-time reset link, without affecting orders.	S	**Implemented**	E2E "every admin page loads"; `customers.test.ts`
FR-ACC-12	A customer shall be able to sign in or sign up with Google, Facebook or Apple when the shop owner has set that provider up and switched it on, and to connect further providers to an existing account.	S	**Implemented**	`social.test.ts`; E2E "social sign-in"
FR-ACC-13	An existing account shall be matched to a provider sign-in only by the provider's own id, or by an email address the provider itself reports as verified (Google, Apple). An unverified email (Facebook), a phone number or a name shall never connect or take over an account.	M	**Implemented**	`social.test.ts`
FR-ACC-14	Each provider sign-in attempt shall be single-use, expire in 10 minutes, be tied to the browser that began it by a cookie (SameSite=None where Apple's cross-site form post requires it), carry a state value, a nonce and, for Google, a PKCE challenge, and refuse an ID token of another issuer, audience, expiry or nonce.	M	**Implemented**	`social.test.ts`; E2E "social sign-in"
FR-ACC-15	A first-time provider sign-in shall ask for a phone number before the account is created, because deliveries are arranged by phone; a phone number already on an account shall not be linked automatically.	M	**Implemented**	`social.test.ts`
FR-ACC-16	The shop shall publish a privacy policy (`/privacy`), terms of service (`/terms`) and data-deletion instructions (`/data-deletion`), linked from every page's footer, using the business name and contact email set by the owner.	M	**Implemented**	E2E "the privacy policy, terms and data-deletion pages"
FR-ACC-17	A customer whose account was made with Google, Facebook or Apple and who has not chosen a password shall be able to delete their account by typing DELETE while signed in; a customer with a password shall be asked for it.	M	**Implemented**	`social.test.ts`
FR-ACC-10	Registration shall verify the phone number or email (for example with a one-time code).	S	**Not implemented**	–
FR-ACC-11	Sign-in, sign-up and reset attempts shall be rate-limited (sign-in 15 per place and 6 per account per 15 minutes; sign-up 10 per hour; reset 6 per place and 3 per account per hour).	M	**Implemented**	`throttle.test.ts`

4.5 Checkout and orders (FR-ORD)

ID	Requirement	Pri	Status	Verified by
FR-ORD-01	Checkout shall require a signed-in customer and send a guest to sign in, returning to checkout with the cart intact.	M	**Implemented**	E2E "checkout asks a guest to sign in"
FR-ORD-02	Checkout shall collect recipient name, phone for the rider, optional email, delivery area, shipping method, address (min 5 characters), optional landmark and note, and the customer's notification choices, validating each on the server.	M	**Implemented**	`admin.test.ts` ; E2E
FR-ORD-03	The customer shall be able to choose a saved address and optionally save a new one.	S	**Implemented**	Inspection
FR-ORD-04	Placing an order shall create it with status *Awaiting payment*, a unique number `UKG-YYYY-NNNNNN` , a unique payment reference, its lines and a first history event, in one transaction.	M	**Implemented**	`admin.test.ts` ("creates an order that is paid exactly once")
FR-ORD-05	An order shall have one of these statuses: Awaiting payment, Payment received, Buying from the UK shop, Bought from the UK shop, Received at our UK address, On its way to Ghana, Clearing customs in Ghana, Out for delivery, Delivered, Cancelled, Refunded.	M	**Implemented**	`order-status.test.ts`
FR-ORD-06	Only a confirmed payment shall set *Payment received*; staff shall not be able to mark an order paid.	M	**Implemented**	`order-status.test.ts` ; `admin.test.ts`
FR-ORD-07	Staff shall be able to move an order one step forward at a time, cancel it up to *Buying from the UK shop*, and mark a paid and cancelled order *Refunded*; other moves shall be refused.	M	**Implemented**	`order-status.test.ts`
FR-ORD-08	Every status change shall be recorded as an event with an optional note, which the customer sees.	M	**Implemented**	E2E "customer sees the status change"
FR-ORD-09	Staff shall be able to add and remove tracking entries at four stages (UK shop dispatch, received at our UK address, UK-to-Ghana shipment, delivery in Ghana), each with carrier, reference, link and note, visible to the customer.	S	**Implemented**	Inspection; E2E (tracking present)
FR-ORD-10	A visitor shall be able to look up an order with its number and the phone or email used, with failures rate-limited (8 per 15 minutes).	S	**Implemented**	`admin.test.ts` ; `throttle.test.ts`
FR-ORD-11	Staff shall be able to list orders (search by number, name or phone; filter by status), open one, and see items, delivery details, price breakdown, payments and history.	M	**Implemented**	E2E "every admin page loads"
FR-ORD-12	Staff shall be able to record the real cost of an order (shop price, UK delivery, purchase rate, freight, Ghana delivery, payment fees, other) and see the margin.	S	**Implemented**	`admin-logic.test.ts`
FR-ORD-13	Staff with the right shall be able to export orders as CSV, with the action logged.	S	**Implemented**	E2E "support account cannot download"; inspection
FR-ORD-14	A customer shall be able to cancel their own order before it is bought.	C	**Not implemented (customers ask; staff cancel)**	–

ID	Requirement	Pri	Status	Verified by
FR-ORD-15	The system shall issue a receipt or invoice document for each paid order.	S	**Not implemented (the order page shows the breakdown)**	–
FR-ORD-16	The system shall limit orders to 10 per visitor per hour.	M	**Implemented**	Inspection

4.6 Payments (FR-PAY)

ID	Requirement	Pri	Status	Verified by
FR-PAY-01	The system shall support Stripe, Paystack and Flutterwave; any number may be switched on and the customer chooses among those on.	M	**Implemented**	`payments.test.ts`
FR-PAY-02	The customer shall pay on the gateway's hosted page; the system shall never receive or store card data.	M	**Implemented**	Inspection
FR-PAY-03	The charge amount and currency shall be taken from the stored order (cedis for Paystack and Flutterwave; pounds or cedis for Stripe by setting).	M	**Implemented**	`payments.test.ts`
FR-PAY-04	An order shall be marked paid only when the gateway confirms success and the **amount and currency match** the order. A mismatch shall not mark it paid and shall be flagged to staff.	M	**Implemented**	`payments.test.ts`
FR-PAY-05	Webhook signatures shall be verified on the raw body; for shared-secret gateways the payment shall be re-checked with the gateway; each event shall be applied once; bodies over 1 MB shall be refused; processing errors shall return 500 so the gateway retries.	M	**Implemented**	`payments.test.ts`
FR-PAY-06	The customer's return from the gateway shall also be confirmed with the gateway (rate-limited).	M	**Implemented**	`payments.test.ts`
FR-PAY-07	A failed or abandoned payment shall leave the order awaiting payment so the customer can retry from the payment page.	M	**Implemented**	Inspection
FR-PAY-08	A pretend payment page shall exist only when explicitly enabled and never by default in production.	M	**Implemented**	E2E (uses it); inspection
FR-PAY-09	The system shall start a refund with the gateway.	C	**Not implemented (staff refund in the gateway dashboard, then mark *Refunded*)**	–

4.7 Notifications (FR-MSG)

ID	Requirement	Pri	Status	Verified by
FR-MSG-01	The system shall send order updates by SMS, WhatsApp and email through providers chosen and switched on by the admin.	M	**Implemented**	`outbox.test.ts`
FR-MSG-02	Which statuses use which channels shall be configurable (defaults: all channels for payment received, UK address, shipped, out for delivery, delivered, cancelled, refunded; email only for buying, bought, customs; none for awaiting payment).	S	**Implemented**	`outbox.test.ts`
FR-MSG-03	A message shall be sent only if the channel's provider is on and configured, the rule allows it, the customer chose that channel, and a valid address exists.	M	**Implemented**	`outbox.test.ts`

ID	Requirement	Pri	Status	Verified by
FR-MSG-04	Messages shall be queued in an outbox, sent immediately, and retried up to three times; failures shall wait for staff with the reason.	M	**Implemented**	`outbox.test.ts`
FR-MSG-05	Phone numbers shall be normalised to international form (Ghana `0…` to `+233…`).	M	**Implemented**	`phone.test.ts`
FR-MSG-06	The system shall send password-reset messages and link-order quote messages (email first, then SMS).	M	**Implemented**	`templates.test.ts` ; `link-orders.test.ts`
FR-MSG-07	Staff shall see a log of messages with status and error, recipients partly hidden, and be able to retry or send queued messages now.	S	**Implemented**	E2E (page loads); inspection
FR-MSG-08	WhatsApp messages shall use approved template variables (name, order number, update).	M	**Implemented**	`outbox.test.ts`

4.8 Reviews and wishlist (FR-REV)

ID	Requirement	Pri	Status	Verified by
FR-REV-01	Only a customer with a **delivered** order for the item shall be able to review it, once.	M	**Implemented**	`discovery.test.ts` ("reviews")
FR-REV-02	A review shall have a rating of 1 to 5, an optional title (up to 100 characters) and body (up to 2,000), shown as first name and last initial.	M	**Implemented**	Inspection
FR-REV-03	Staff shall be able to hide or publish reviews; only published reviews count towards the rating and appear.	S	**Implemented**	`discovery.test.ts` ("reviews")
FR-REV-04	Signed-in customers shall be able to save items to a wishlist and see it in their account.	S	**Implemented**	E2E "wishlist saves an item"

4.9 Link orders (FR-LNK)

ID	Requirement	Pri	Status	Verified by
FR-LNK-01	A visitor shall be able to submit a product link from the home page box, the header, the search box and a request page, with quantity, size/colour notes and item type; the system shall attach the request to the signed-in customer.	M	**Implemented**	E2E "a link request is quoted, paid for…"
FR-LNK-02	The system shall read the link's name and price when the shop's `robots.txt` allows, show the price in pounds and cedis, and say when an item is already listed.	S	**Implemented**	`run.test.ts` ; E2E (faked answer)
FR-LNK-03	A found link shall continue in one step to the customer's price and payment; the price shall be read again on the server and never taken from the browser.	S	**Implemented**	`link-submit.test.ts` ; E2E "a found link goes straight…"
FR-LNK-04	Staff shall be able to **quote** a request with the UK price of one item, its weight, a validity period (1 to 30 days) and a note; the quote shall create a private pay link that is messaged to the customer and shown to staff. A quote can be changed until the customer orders without changing the link.	M	**Implemented**	`link-orders.test.ts` ; E2E
FR-LNK-05	The pay link shall use a long random token, show the full cost, require the customer to be signed in, expire after its validity, and create **at most one order**.	M	**Implemented**	`link-orders.test.ts`

ID	Requirement	Pri	Status	Verified by
FR-LNK-06	Paying a quote shall create an ordinary order (same pricing, minimum order, statuses, tracking, messages, refunds and costs) linked back to the request.	M	**Implemented**	`link-orders.test.ts`
FR-LNK-07	The system shall quote by itself, when the admin allows it, from (a) the price read from the shop's own page (default on) or (b) the customer-typed price plus an admin-set safety margin (default off), only up to an admin-set limit per item (default £150), using a weight from an admin-set item-type list. Otherwise the request shall wait for staff.	S	**Implemented**	`link-auto.test.ts` ; E2E "automatic quotes…"
FR-LNK-08	A link order shall show staff how the price was found (shop page, customer-typed plus margin, or staff-quoted) as a reminder to verify before buying.	S	**Implemented**	Inspection
FR-LNK-09	Customers shall see their link requests and their state (checking, price ready, expired, ordered) in their account.	S	**Implemented**	E2E
FR-LNK-10	Link requests shall be rate-limited to 6 per visitor per hour and previews to 12 per 10 minutes.	M	**Implemented**	Inspection
FR-LNK-11	The system shall verify the real shop price automatically at purchase time.	C	**Not implemented (staff verify by hand; a banner reminds them)**	–
FR-LNK-12	The system shall recognise Amazon product links (including short links and shared text) by their ASIN, turn an Amazon UK link into its clean `amazon.co.uk/dp/ASIN` form, and refuse other Amazon stores with a message offering the UK link. The shop shall never request an Amazon page itself; the customer reads the price on Amazon and types it, and staff verify before buying.	S	**Implemented**	`amazon-links.test.ts` ; E2E "ordering from Amazon UK…"
FR-LNK-13	The search page shall offer a hand-off to Amazon UK's own search for the same words, and an Amazon page (`/amazon`) shall explain the steps and offer a one-click bookmarklet that opens the request form pre-filled from the page the customer is viewing.	S	**Implemented**	E2E
FR-LNK-14	The shop shall be installable as a phone app and appear in the phone's Share menu (web app manifest with a share target), so a customer can share an Amazon product from the Amazon app straight into a request.	C	**Implemented**	E2E
FR-LNK-15	A customer shall be able to request an item by describing it (at least 3 characters) without a link. Such a request shall never be quoted automatically; staff shall be offered searches for it and can attach the link they find when quoting.	S	**Implemented**	`link-submit.test.ts` ; E2E "an item described in words…"

4.10 Catalogue management (FR-CAT)

ID	Requirement	Pri	Status	Verified by
FR-CAT-01	Staff with the right shall be able to create, edit, show/hide and delete shops (name, department, tagline, description, website, colour, order, logo by upload or link). Deleting a shop shall remove its items, sources and cart/wishlist entries but never alter placed orders; confirmation shall be required.	M	**Implemented**	`shop-delete.test.ts` ; `shop-logo.test.ts` ; E2E
FR-CAT-02	Staff shall be able to create and edit items (name, brand, category, description, UK price, weight, size/colour options, source link, photo upload or link, shown/hidden).	M	**Implemented**	`admin.test.ts` ; E2E

ID	Requirement	Pri	Status	Verified by
FR-CAT-03	Uploaded images shall be accepted by file content (JPEG, PNG, WebP, GIF), up to 4 MB, stored under random names, and served with a fixed type, `nosniff` and a sandbox policy.	M	**Implemented**	`uploads.test.ts` ; E2E "fake images are refused"
FR-CAT-04	A new database shall contain only the shop **eBay UK**; made-up sample data shall be created only for tests or when explicitly enabled.	M	**Implemented**	`seed.test.ts`
FR-CAT-10	The system shall support catalogue sources of these kinds: product feed (CSV/JSON), eBay API, Diffbot Product API, Shopify shop, WooCommerce shop, shop website (sitemap and product pages), file import, pasted links.	M	**Implemented**	`run.test.ts` , `ebay.test.ts` , `shopify.test.ts` , `woocommerce.test.ts` , `file-import.test.ts`
FR-CAT-11	A source shall not be switched on until staff confirm they have checked the shop's terms or hold a licence.	M	**Implemented**	`run.test.ts` ; E2E "sources need confirmed permission"
FR-CAT-12	Staff shall be able to preview a source without saving, run it now, switch it on or off, set its schedule and rules, and remove it keeping or removing its products.	M	**Implemented**	`run.test.ts` ; E2E
FR-CAT-13	The system shall run due sources automatically (built-in scheduler, 10-minute tick; each source default every 24 hours) or via an external cron endpoint.	M	**Implemented**	`run.test.ts` (due sources); inspection
FR-CAT-14	New items shall be published automatically unless held; changes of price, stock or photo shall update live items; items missing from a complete feed shall be hidden only if at least half remain; items not refreshed within the source's stale window (default 14 days) shall be hidden.	M	**Implemented**	`run.test.ts`
FR-CAT-15	Items with a price under 50p or over £10,000, a name under 3 characters, a name that looks like a warranty, protection plan or fee rather than a product, no link back, or a price move above the source's limit (default 40%) shall be held for staff review; staff shall be able to approve or reject singly or in bulk.	M	**Implemented**	`run.test.ts`
FR-CAT-16	Only prices in pounds shall be accepted.	M	**Implemented**	`parse.test.ts` ; `shopify.test.ts`
FR-CAT-17	Feed addresses (which often hold keys) shall be stored encrypted and shown truncated.	M	**Implemented**	`run.test.ts` ; `secrets.test.ts`
FR-CAT-18	A Shopify or WooCommerce source shall read the shop's public product list only if `robots.txt` allows, the shop's currency is GBP, and the shop is a Shopify (or, for WooCommerce, a Store API) shop; sizes and colours shall become item options; products whose variants differ in price shall be skipped.	M	**Implemented**	`shopify.test.ts` , `woocommerce.test.ts`
FR-CAT-19	A file-import source shall accept an uploaded CSV or JSON, apply the same checks as a feed, update on re-upload, and never remove products.	S	**Implemented**	`file-import.test.ts` ; E2E
FR-CAT-20	An eBay source shall sign in with the admin's keys, search each listed query, accept only new, fixed-price, UK-located GBP listings, and never treat a missing result as removal.	M	**Implemented**	`ebay.test.ts`
FR-CAT-21	A Diffbot source shall read each listed product page through Diffbot's Product API using the admin's token, check the shop's `robots.txt` first and send nothing for a disallowed page, accept only prices in GBP, stop on a bad token, exhausted credits or rate limit, and never treat the list as complete.	M	**Implemented**	`diffbot.test.ts`

ID	Requirement	Pri	Status	Verified by
FR-CAT-30	The importer shall identify itself honestly (`ShopCatalogBot` , with this site's address in its user agent) and obey `robots.txt` and any `Crawl-delay` , with at least 2 seconds between requests to one shop.	M	**Implemented**	`net.test.ts`
FR-CAT-31	The importer shall fetch only http/https on ports 80 and 443, never reach private or internal addresses (also checked at name resolution), follow at most three redirects (each re-validated), and enforce size and time limits.	M	**Implemented**	`net.test.ts`
FR-CAT-32	When a shop answers 401, 403, 429 or 451, or shows a verification page, the importer shall stop, pause that source for 24 hours and tell staff.	M	**Implemented**	`net.test.ts` ; `run.test.ts`
FR-CAT-33	The importer shall never evade a block: no changed identity, rotating addresses, or CAPTCHA solving.	M	**Implemented (by design)**	Inspection; `net.test.ts`
FR-CAT-34	Only one run per source at a time (lock expiring after 30 minutes).	M	**Implemented**	`run.test.ts`

4.11 Administration (FR-ADM)

ID	Requirement	Pri	Status	Verified by
FR-ADM-01	The dashboard shall show, for the last 7, 30 or 90 days, revenue, orders, average order, service charge earned, new customers (each compared with the previous period), sales by day, orders in progress by status, revenue composition, margin, top shops, items and areas, customers, the exchange rate, integrations status, latest orders and recent activity.	M	**Implemented**	`admin-logic.test.ts` ; E2E
FR-ADM-02	The dashboard shall show a prioritised "needs attention" list: paid orders needing a refund, payments with wrong amount, paid orders to buy, new link requests, stale orders (5+ days), imported items to review, stopped sources, failed messages, unpaid orders over 24 hours, and an exchange-rate gap.	M	**Implemented**	`admin-logic.test.ts`
FR-ADM-03	Staff shall be able to set the exchange rate and markup, the service charge, the minimum order, the site name and the support WhatsApp number, with changes to the rate recorded in a history.	M	**Implemented**	`admin-logic.test.ts` ("exchange rate"); `admin.test.ts` ; E2E
FR-ADM-04	Staff shall be able to create, edit, hide and price shipping methods and rate cards, with validation (for example ascending brackets).	M	**Implemented**	`admin-parse.test.ts` ; `admin.test.ts`
FR-ADM-05	Staff shall be able to create, edit, hide and delete delivery areas; the last active area shall not be deletable; orders shall keep the area name and fee charged.	M	**Implemented**	`shop-delete.test.ts` ; E2E
FR-ADM-06	Staff shall be able to choose one of 15 colour themes for the whole shop.	S	**Implemented**	`themes.test.ts` ; E2E
FR-ADM-07	Staff shall be able to enter, switch on, test and clear provider keys; keys shall be encrypted at rest, shown masked, never logged, and overridden by environment settings; a readiness summary shall list what is missing.	M	**Implemented**	`integrations.test.ts` ; `secrets.test.ts`
FR-ADM-08	The admin shall record every administrative action and sign-in in an activity log naming the person, searchable by staff with the right.	M	**Implemented**	E2E (audit actors checked); inspection

ID	Requirement	Pri	Status	Verified by
FR-ADM-09	The exchange-rate feed policy shall be manual, suggest or automatic, with a maximum automatic move (default 5%) and an alert gap (default 3%); automatic moves beyond the limit shall wait for staff.	S	**Implemented**	`fx-api.test.ts`
FR-ADM-10	The admin shall not require a technical user: plain language, confirmation for destructive actions (delete shop, area, account, source), and clear errors.	S	**Implemented**	Inspection; E2E

4.12 Staff accounts and access control (FR-STF)

ID	Requirement	Pri	Status	Verified by
FR-STF-01	There shall be one **super admin**, authenticated by the server-held `ADMIN_PASSWORD` at a separate developer sign-in, not stored in the database, who can do everything and is the only person able to manage staff accounts.	M	**Implemented**	`admin-guards.test.ts` ; E2E
FR-STF-02	The super admin shall be able to create staff accounts (name, email, first password, role), see them, change name, role and rights, switch them off and on, reset their password, and delete them (with confirmation).	M	**Implemented**	`admin-users.test.ts` ; E2E
FR-STF-03	Each staff account shall hold a set of **rights**; roles (Manager, Operations, Customer support, Catalogue editor, Finance, Read-only) shall be presets, and the super admin shall be able to tick exact rights ("Custom"). A change right shall imply the matching view right.	M	**Implemented**	`permissions.test.ts`
FR-STF-04	The right to manage staff accounts shall not be grantable to staff.	M	**Implemented**	`permissions.test.ts` ; `admin-guards.test.ts`
FR-STF-05	**Every** admin page, admin server action and the orders download shall check the specific right on the server for each request; the menu shall show only permitted pages and other pages shall show "No access".	M	**Implemented**	`admin-guards.test.ts` ; E2E "support account sees and does only what its role allows"
FR-STF-06	A new or reset staff account shall have to choose its own password (at least 10 characters) before any other admin page opens.	M	**Implemented**	`admin-users.test.ts` ; E2E
FR-STF-07	Switching an account off, changing its rights or resetting its password shall end its open sessions at once.	M	**Implemented**	`admin-users.test.ts` ; E2E "switching a staff account off…"
FR-STF-08	Staff shall not be able to give themselves more access; a staff member shall be able to change only their own password.	M	**Implemented**	`admin-guards.test.ts`
FR-STF-09	Admin sign-in shall be rate-limited (5 failures per place and 8 per email per 15 minutes), with the same timing for unknown emails.	M	**Implemented**	`auth.test.ts` ; `admin-users.test.ts`
FR-STF-10	Staff actions shall be recorded under the person's name and email.	M	**Implemented**	E2E (audit actors)
FR-STF-11	Sessions shall use signed, HttpOnly, SameSite cookies, 8 hours, tamper-evident and bound to the account's session version.	M	**Implemented**	`auth.test.ts`
FR-STF-12	Staff sign-in shall support two-step verification.	S	**Not implemented**	–

ID	Requirement	Pri	Status	Verified by
FR-STF-13	New staff shall receive an email invitation to set a password.	C	**Not implemented (the super admin gives the first password)**	–

4.13 System and operations (FR-SYS)

ID	Requirement	Pri	Status	Verified by
FR-SYS-01	`/api/health` shall report whether the server is up and the database readable (`{"ok":true}` or 503) without exposing details.	M	**Implemented**	Inspection; E2E
FR-SYS-02	Cron endpoints (`/api/cron/messages` , `/fx` , `/ingest`) shall answer 404 until a secret is set and then require a bearer secret.	M	**Implemented**	E2E "unsigned webhook and unauthenticated cron are refused"
FR-SYS-03	The database schema shall be created and migrated automatically at start without losing data, including rebuilding a table when a rule must change.	M	**Implemented**	`source-kinds-migration.test.ts` ; `shop-logo.test.ts` ; `link-orders.test.ts`
FR-SYS-04	Deleting all shops shall not cause the starter data to be re-created at restart.	M	**Implemented**	`shop-delete.test.ts`
FR-SYS-05	The system shall serve security headers (frame denial, no-sniff, referrer policy, permissions policy, HSTS) and a `robots.txt` that disallows private areas.	M	**Implemented**	Inspection
FR-SYS-06	The public address shall come from configuration, never from request headers.	M	**Implemented**	Inspection (`app-url.ts`)
FR-SYS-07	Secrets shall never appear in logs or pages.	M	**Implemented**	`integrations.test.ts` ; inspection
FR-SYS-08	A page shall explain the importer to shop owners (`/bot`).	S	**Removed** (owner's decision; the importer's user agent links to the site's home page instead)	E2E checks the page is gone

---

5. Verification and traceability

5.1 Verification methods

Method	Meaning
Test	An automated check: a unit or database test (`npm test` , 321 tests in 41 files) or a step of the end-to-end browser run (`npm run e2e` , 40 steps).
Inspection	Reading the code or screen. Used where a check cannot be automated sensibly (for example wording).
Demonstration	Doing it by hand on the live site. Required for real payments, real shop pages and real message delivery (see 5.4).

5.2 Traceability: requirement areas to tests

Area	Requirement IDs	Automated checks
Browsing and search	FR-BRW	`browse.test.ts`, `discovery.test.ts`, `department-icons.test.ts`, E2E (home, search, suggestions)
Cart	FR-CRT	`admin.test.ts`, E2E (add to cart, cart total)
Pricing	FR-PRC	`pricing.test.ts`, `admin.test.ts`, `admin-parse.test.ts`, E2E (checkout total equals cart)
Accounts	FR-ACC	`customers.test.ts`, `customer-session.test.ts`, `password.test.ts`, `account-orders.test.ts`, `throttle.test.ts`, E2E (register, account)
Orders	FR-ORD	`order-status.test.ts`, `admin.test.ts`, `admin-logic.test.ts`, `account-orders.test.ts`, E2E (checkout, pay, admin moves order)
Payments	FR-PAY	`payments.test.ts`, E2E (demo payment)
Notifications	FR-MSG	`outbox.test.ts`, `templates.test.ts`, `phone.test.ts`
Reviews and wishlist	FR-REV	`discovery.test.ts`, `account-orders.test.ts`, E2E (wishlist)
Link orders	FR-LNK	`link-orders.test.ts`, `link-auto.test.ts`, `link-submit.test.ts`, E2E (link request to order, automatic quotes, found-link flow)
Catalogue	FR-CAT	`ingest/net.test.ts`, `parse.test.ts`, `field-map.test.ts`, `run.test.ts`, `ebay.test.ts`, `diffbot.test.ts`, `shopify.test.ts`, `woocommerce.test.ts`, `file-import.test.ts`, `source-kinds-migration.test.ts`, `seed.test.ts`, `shop-delete.test.ts`, `shop-logo.test.ts`, `uploads.test.ts`, `secrets.test.ts`, E2E (sources, shops, uploads)
Administration	FR-ADM	`admin-logic.test.ts`, `admin.test.ts`, `integrations.test.ts`, `fx-api.test.ts`, `themes.test.ts`, `csv.test.ts`, E2E (every admin page, themes, delivery areas)
Staff and access	FR-STF	`admin-users.test.ts`, `permissions.test.ts`, `auth.test.ts`, `admin-guards.test.ts`, E2E (staff account journey)
System	FR-SYS	`source-kinds-migration.test.ts`, `shop-delete.test.ts`, E2E (health, cron, webhooks)
Phone layouts	IR-UI-01 to 03	E2E (shop fits a phone, admin fits a phone)

5.3 End-to-end acceptance journeys (automated)

The browser run is the system's **acceptance test**. Each step below must pass for a release:

1. Protected pages redirect guests; unsigned webhooks and unauthenticated cron calls are refused.
2. Search suggests items as you type; results page finds the item.
3. A guest adds two trainers to the cart; the cart total is right; checkout asks them to sign in; they register and return with the cart intact.
4. They pay (demo payment) and the order shows in their account with an updates entry.
5. The admin signs in (wrong password refused); every admin page loads; the admin moves the order; the customer sees the change.
6. Changing the service charge and exchange rate reaches the storefront; an invalid tier setup is rejected clearly.

7. Catalogue sources need permission and refuse internal addresses; a source can be created and removed; the eBay, Diffbot, Shopify and file-import forms work.
8. A shop and a delivery area can be added and deleted only after confirmation; a shop logo uploads and shows; fake images are refused.
9. A colour theme chosen in the admin reaches the storefront.
10. A link request is quoted, paid and appears as an ordinary order; automatic quotes price an ordinary request and leave a dear one for staff; a found link goes straight to the customer's request.
11. A staff account is created, must change its password, sees only its role's pages, is blocked elsewhere, cannot change orders or download exports, and is signed out when switched off.
12. The shop and the admin fit a phone with no sideways scrolling.

5.4 Not verified automatically (must be demonstrated)

- Real payments with each gateway, and real webhook delivery.
- Reading a real shop's page or feed (the test environment cannot reach retailer sites).
- Real SMS, WhatsApp and email delivery and template approvals.
- Behaviour on real iPhone and Android devices.
- Behaviour under load; recovery from a restored backup. See Testing → manual checklist.

5.5 Acceptance criteria for release

All automated checks pass (`npm run lint && npm run typecheck && npm test && npm run e2e`); the manual checklist is completed; a backup has been taken and restored once; at least one payment gateway is configured and has processed a real small order end to end.

---

6. Non-functional requirements

Status for these is **Implemented**, **Partial** or **Target** (a goal that has not been measured).

6.1 Performance

ID	Requirement	Pri	Status
NFR-PER-01	Pages shall render from live data on every request; list and product pages should respond within 1 second at the 95th percentile for one small shop (up to a few thousand items and a few hundred orders a day) on the Starter instance.	S	**Target (not load-tested; the database is local and queries are indexed)**
NFR-PER-02	Search suggestions shall respond fast enough to feel instant and shall be limited to 120 requests per visitor per minute.	S	**Implemented (limit); latency not measured**
NFR-PER-03	The catalogue importer shall not slow shopping: runs are sequential, paced, size-limited, and a source run never holds a database transaction across a network wait.	M	**Implemented**
NFR-PER-04	Pages should be light for mobile data: no external fonts, scripts or trackers; images lazy-loaded.	S	**Implemented**

6.2 Reliability and availability

ID	Requirement	Pri	Status
NFR-REL-01	Order creation and payment confirmation shall be atomic and idempotent: no half-created orders; a repeated webhook shall have no further effect.	M	**Implemented**
NFR-REL-02	A messaging or provider failure shall never break an order; messages shall retry.	M	**Implemented**
NFR-REL-03	The system shall start cleanly after a restart or redeploy, applying schema changes automatically.	M	**Implemented**
NFR-REL-04	The target availability is that of a single always-on instance (no redundancy). Planned maintenance interruption is a few minutes per deploy.	S	**Partial (no failover by design; see DC-1)**
NFR-REL-05	The importer shall fail safe: a bad feed shall never wipe the catalogue (removal only from complete feeds and only if at least half the items remain).	M	**Implemented**

6.3 Security (summary; full detail in Security and privacy)

ID	Requirement	Pri	Status
NFR-SEC-01	Passwords shall be hashed with scrypt; session tokens and reset tokens stored hashed.	M	**Implemented**
NFR-SEC-02	Stored secrets (provider keys, feed addresses) shall be encrypted with AES-256-GCM.	M	**Implemented**
NFR-SEC-03	Cookies shall be HttpOnly, SameSite=Lax and Secure in production; customer cookies use the `__Host-` prefix.	M	**Implemented**
NFR-SEC-04	All database access shall use bound parameters.	M	**Implemented**
NFR-SEC-05	Authentication and sensitive actions shall be rate-limited.	M	**Implemented**
NFR-SEC-06	Uploaded files shall be validated by content and served so they cannot execute.	M	**Implemented**
NFR-SEC-07	The server shall not be usable to reach internal networks (SSRF protection on every outbound fetch it makes on a user's behalf).	M	**Implemented**
NFR-SEC-08	Access shall be least-privilege by right, enforced on the server per request, and every admin action audited.	M	**Implemented**
NFR-SEC-09	A Content-Security-Policy shall protect the main pages.	S	**Not implemented**
NFR-SEC-10	Rate limits shall be shared across instances.	C	**Not implemented (single instance only)**

6.4 Usability and accessibility

ID	Requirement	Pri	Status
NFR-USE-01	A first-time shopper shall be able to find an item, see its full cost and pay without instructions.	M	**Implemented (design); not user-tested**
NFR-USE-02	Destructive admin actions shall need explicit confirmation.	M	**Implemented**
NFR-USE-03	Error messages shall say what is wrong and what to do.	S	**Implemented**
NFR-USE-04	Forms shall have labelled fields, visible focus and keyboard operation.	S	**Implemented**
NFR-USE-05	The site shall meet WCAG 2.1 AA.	S	**Partial (no formal audit)**
NFR-USE-06	The interface shall be in English; amounts in GHS and GBP.	M	**Implemented**

6.5 Compatibility and portability

ID	Requirement	Pri	Status
NFR-CMP-01	Support current Chrome, Edge, Firefox and Safari (desktop and mobile).	M	**Partial (tested in Chrome and an emulated phone; other browsers by design only)**
NFR-CMP-02	Run in a Docker container on any host with a persistent disk (Render, Fly.io, a VPS).	M	**Implemented**
NFR-CMP-03	The database shall be a single portable file.	M	**Implemented**

6.6 Maintainability and operability

ID	Requirement	Pri	Status
NFR-MNT-01	Code shall pass lint and type checks; logic shall have automated tests; a guard test shall prevent unprotected admin pages.	M	**Implemented**
NFR-MNT-02	Pricing, access checks and the network layer shall each live in one place.	M	**Implemented**
NFR-MNT-03	Documentation shall describe use, operation, architecture, data, security and requirements.	M	**Implemented (this set)**
NFR-OPS-01	A health endpoint, logs without secrets, and an in-admin attention list shall make problems visible.	M	**Implemented**
NFR-OPS-02	Backups shall be possible while running (SQLite backup API) and restorable.	M	**Implemented (procedure documented; the schedule is the operator's)**

6.7 Backup and recovery targets

ID	Requirement	Status
NFR-BKP-01	The operator shall back up the database and uploads at least daily and keep a copy off the server. Recovery point equals the backup interval.	**Operator duty (not automated by the software)**
NFR-BKP-02	A restore shall be rehearsed before launch. Target recovery time: within a working day.	**Operator duty**

6.8 Capacity and scalability

The system is sized for **one small business**: one instance, one disk, hundreds of orders a day at most. It is **not** designed for horizontal scaling. Growing beyond this requires replacing SQLite with a client-server database, moving timers to a worker, and sharing rate limits (see Appendix B).

6.9 Legal and regulatory (not provided by software)

The operator is responsible for consumer-protection terms and refund policy, VAT and import duty, payment-service licensing, data-protection registration and lawful handling of personal data (Ghana Data Protection Act and, where relevant, UK/EU rules), and for the right to display imported product data and images. The software provides tools (audit log, account deletion, access control) but not legal compliance.

7. Data requirements

ID	Requirement	Status
DR-01	Persistent data shall live in one SQLite database (WAL mode, foreign keys on) plus an uploads folder next to it. See Data model.	**Implemented**
DR-02	Money shall be stored as integer minor units.	**Implemented**
DR-03	An order shall be self-contained: it stores customer, delivery, area, shipping, amounts, rate and each item as at purchase; deleting a shop, item or area shall not change it.	**Implemented**
DR-04	Secrets shall be encrypted; passwords and tokens hashed.	**Implemented**
DR-05	Times shall be stored in UTC.	**Implemented**
DR-06	The audit log shall be append-only through the application.	**Implemented**
DR-07	Schema changes shall be forward-only, additive and applied automatically; a rule change shall rebuild the table keeping rows.	**Implemented**
DR-08	Personal data shall be limited to what is needed (name, phone, optional email, addresses, orders, sessions, staff details).	**Implemented**
DR-09	Retention: orders, link requests and the activity log are kept until the operator removes them; the system has no automatic deletion.	**Partial (operator-defined; see Appendix B)**
DR-10	A customer's personal data shall be exportable on request.	**Not implemented (done by hand from the database)**

Entities (all in the Data model): shops, products, carts and lines, orders with items, events, tracking, costs and payments, link requests, customers with sessions, addresses, resets, wishlist and reviews, staff accounts, audit log, settings and integration settings, shipping methods, delivery zones, exchange-rate history, messages, webhook events, catalogue sources, imported items and runs.

Indicative volumes (planning assumptions): up to ~5,000 items, ~500 orders a day, ~50,000 customers, ~1 GB total for the first year including photos. Photos are the main growth.

---

8. Business rules

ID	Rule
BR-01	**Customer total** = items + service charge + shipping to Ghana + delivery in Ghana. Duties and taxes in Ghana are excluded.
BR-02	**Effective rate** = rate × (1 + markup%). Items are converted at the effective rate.
BR-03	**Minimum order**: items only, in pounds; default £10. Applies to link orders too.
BR-04	**Order status path**: Payment received → Buying from the UK shop → Bought from the UK shop → Received at our UK address → On its way to Ghana → Clearing customs → Out for delivery → Delivered.
BR-05	**Only a gateway-confirmed payment** with matching amount and currency makes an order *Payment received*. Staff cannot mark an order paid.
BR-06	Staff move an order **one step forward**; they may **cancel** up to *Buying from the UK shop*; a **paid and cancelled** order may be marked **Refunded** after the refund is made in the gateway.
BR-07	**Nothing is bought before payment is confirmed.**
BR-08	A **review** is allowed only for an item the customer received (order *Delivered*), once per item.

ID	Rule
BR-09	**No discount claims**: customers see only the shop's current price (in pounds and cedis). The shop never shows a "was" price, a percentage off, or a saving.
BR-10	**Imported prices** are accepted only in pounds; an item is held for review if the price is under 50p, over £10,000, jumps by more than the source's limit (default 40%), the name is under 3 characters or there is no link back.
BR-11	**Disappearing items**: hidden only when missing from a *complete* feed and at least half of the known items remain; items not refreshed within the stale window (default 14 days) are hidden.
BR-12	**A source needs permission**: it cannot be switched on until the operator confirms the shop's terms or a licence.
BR-13	**The importer stops on refusal** (401, 403, 429, 451 or a verification page) and pauses that source for 24 hours; it never tries to evade.
BR-14	**A quote** is held for the days set (1 to 30, default 3), has one private link, can be changed until used, and produces **at most one order**.
BR-15	**Automatic quotes** apply only when the admin allows, only up to the automatic limit (default £150 per item), and a customer-typed price always carries the safety margin (default 5%).
BR-16	**Deleting a shop or delivery area** never changes placed orders; the **last active delivery area** cannot be deleted.
BR-17	**Staff accounts** are created only by the super admin; a staff member can never grant rights; new or reset accounts must choose their own password; switching off or changing an account signs it out at once.
BR-18	**Notifications** go out only on channels the customer chose, the admin enabled for that status, and a configured provider supports.
BR-19	**Cancelled before shipment** is the customer's window to withdraw; after the UK shop has bought the item it normally cannot be cancelled. (Policy; not enforced by software beyond BR-06.)
BR-20	**Prices shown are guidance until paid**: the server recomputes every amount at order creation.

---

9. Use cases

UC-01 Browse and buy a listed item

- **Actor:** Customer. **Pre:** none (sign-in required at checkout). **Related:** FR-BRW, FR-CRT, FR-PRC, FR-ORD, FR-PAY.
- **Main flow:** 1. Customer finds an item (browse or search). 2. Chooses options and quantity, **Add to cart**. 3. Opens the cart; sees totals in cedis and pounds. 4. **Checkout**: signs in or registers; chooses area and shipping; enters address; chooses notifications. 5. **Place order and pay**; chooses a gateway; pays on its page. 6. The system confirms payment, sets *Payment received*, sends a message. 7. Customer follows tracking.
- **Alternatives:** payment fails → order stays awaiting payment, customer retries. Below minimum → told how much more is needed. Item or option missing → cannot add.
- **Post:** an order exists with status *Payment received*; staff see it as "waiting to be bought".

UC-02 Paste a link and buy (automatic quote)

- **Actor:** Customer. **Related:** FR-LNK-01 to -03, -07.
- **Main flow:** 1. Customer pastes a product link. 2. The system reads the name and price from the shop's page (allowed) and shows pounds and cedis. 3. Customer enters quantity, kind of item and size/colour; taps **Get my full price and pay**. 4. The server re-reads the page, applies the rules, quotes, and opens the quote page. 5. Customer signs in if needed, sees the full cost, chooses delivery, pays. 6. As UC-01 from step 5.

- **Alternatives:** page unreadable or price above the limit → request waits for staff (UC-03). Quote expired → asks for a fresh one.

UC-03 Link request quoted by staff

- **Actors:** Customer, Staff with *Handle link requests*. **Related:** FR-LNK-04 to -06, -08, -09.
- **Main flow:** 1. A request arrives (listed in **Link requests** and on the dashboard). 2. Staff open the link on the shop; confirm price and stock. 3. Staff enter UK price, weight, days and a note; **Send quote**. 4. The customer is messaged a private link and sees it in their account. 5. Customer pays (UC-01 step 5). 6. The request becomes *Ordered* and links to the order.
- **Alternatives:** staff reject the request → customer sees "could not get this item". Staff change the quote → same link keeps working.

UC-04 Fulfil an order

- **Actor:** Staff with *Update orders*. **Related:** FR-ORD-07 to -09, -12, FR-MSG.
- **Main flow:** 1. Staff open a *Payment received* order. 2. Buy the items from the UK shop (outside the system). 3. Mark *Buying…* then *Bought…* with the shop's order reference as a note. 4. When received in the UK, add tracking and mark *Received at our UK address*. 5. On shipping, add tracking and mark *On its way to Ghana*; then *Clearing customs*, *Out for delivery* (with courier reference), *Delivered*. 6. Record real costs to see margin.
- **Post:** the customer saw each step and received the configured messages.

UC-05 Cancel and refund

- **Actor:** Staff with *Update orders*. **Related:** FR-ORD-07, BR-06.
- **Main flow:** 1. Customer asks to cancel before buying. 2. Staff **Cancel** the order. 3. The dashboard shows "needs a refund" (paid orders). 4. Staff refund in the gateway's dashboard. 5. Staff mark the order **Refunded**; the customer is messaged.

UC-06 Fill the catalogue from a source

- **Actor:** Staff with *Manage catalogue sources*. **Related:** FR-CAT-10 to -34.
- **Main flow:** 1. Staff create a source (kind, address or searches), confirm permission. 2. **Check this setup first** to preview. 3. **Create source** and **Run now**. 4. New items go live; odd ones wait in **Import review**. 5. The scheduler refreshes prices and stock and hides stale items.
- **Alternatives:** the shop refuses → the run stops, the source pauses 24 hours and shows why.

UC-07 Manage staff access

- **Actor:** Super admin. **Related:** FR-STF.
- **Main flow:** 1. Super admin signs in at the developer sign-in. 2. Creates an account with a role and first password. 3. The staff member signs in, chooses their own password, and sees only their pages. 4. Later the super admin changes rights, resets the password or switches the account off (immediate sign-out).

UC-08 Set prices and delivery

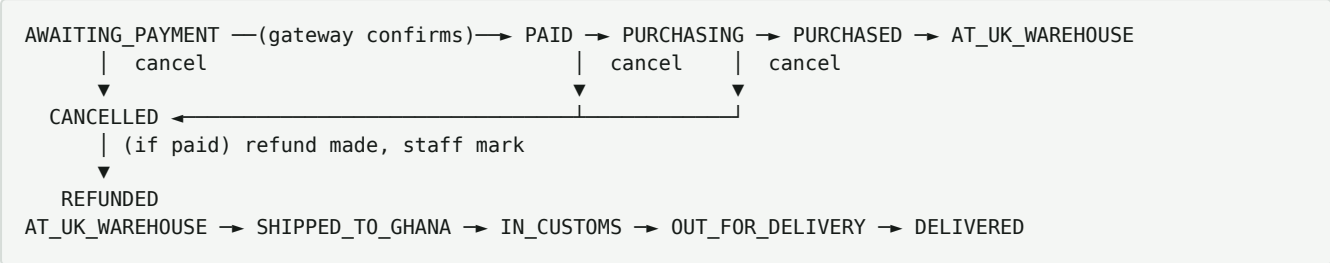
- **Actor:** Staff with *Change prices and delivery*. **Related:** FR-PRC, FR-ADM-03 to -05.
- **Main flow:** 1. Open **Pricing**: set rate, markup, service charge, minimum order. 2. Open **Shipping**: rate cards. 3. Open **Delivery areas**: areas and fees. 4. Check "What customers pay now". 5. Save.

UC-09 Customer recovers a forgotten password

- **Actor:** Customer. **Related:** FR-ACC-04.
- **Main flow:** 1. **Forgot password**. 2. Enters phone or email. 3. Receives a link (email, else SMS) valid 60 minutes. 4. Sets a new password; is signed in; other devices are signed out.
- **Alternative:** no message arrives → staff create a one-time link for the customer.

State models

Order status



Link request: NEW → QUOTED (manually or automatically) → ORDERED (customer paid), or REJECTED; a quote can expire and be re-quoted. **Catalogue source:** off → on (after permission) → running → idle; any refusal → paused 24 hours; remove at any time (not while running). **Catalogue item:** PENDING → PUBLISHED (live) ⇄ HELD (needs review) → REJECTED.

---

10. Constraints, assumptions, dependencies and risks

10.1 Constraints

See 2.5 (DC-1 to DC-6). Additionally: the importer's refusal to evade blocks is a deliberate **business constraint**, not a defect (FR-CAT-33).

10.2 Risks

ID	Risk	Impact	Mitigation in the system	Residual
R-1	A retailer's terms forbid collecting or showing its data, or it blocks the importer.	Legal exposure; missing products.	Permission confirmation, `robots.txt`, stop-on-refusal, affiliate feeds, link orders.	Operator must check terms and licences.
R-2	A link-order price read or typed is wrong (several prices on a page, a typo, an old price).	Margin loss on a paid order.	Quote validity, safety margin, automatic limit, per-order banner, staff verify before buying, cancel and refund.	Verification before buying is manual (FR-LNK-11).
R-3	Exchange-rate movement between quote and purchase.	Margin loss.	Markup on the rate, optional feed with alert and limits, quote expiry.	Operator discipline on the rate.
R-4	Data loss (disk failure, deletion).	Loss of orders and customers.	Backup API, documented procedure, restore rehearsal.	**Backups are the operator's duty.**
R-5	Leak of keys or the super admin password.	Fraud, data exposure.	Encryption at rest, masked display, separate encryption key, rotation runbook, audit log.	No two-step sign-in.
R-6	Payment mismatch or fraud.	Shipping unpaid goods.	Amount and currency verification, signed webhooks, wrong-amount alert.	Gateway-side fraud checks are the gateway's.

ID	Risk	Impact	Mitigation in the system	Residual
R-7	Single instance outage.	Shop unavailable.	Health check, auto-restart by the host.	No redundancy by design.
R-8	Staff misuse of access.	Data or price tampering.	Least-privilege rights, audit log, immediate switch-off.	Trust in the super admin.
R-9	Customs duty, VAT or consumer-law non-compliance.	Fines, disputes.	Total excludes duty and says so.	Needs professional advice (A-5).
R-10	Image or data licensing breaches.	Takedown claims.	Source terms confirmation; images linked from source.	Operator responsibility (A-3).
R-11	Third-party API changes or outages.	Features stop.	Isolated adapters, clear errors, retries, manual fall-backs.	Provider dependence.
R-12	SQLite or in-process timers limit growth.	Scaling ceiling.	Documented single-instance rule.	Re-architecture needed for scale.

Appendix A: Requirement status summary

Counted from the requirement tables above (functional, interface, non-functional and data requirements).

Area	Implemented	Partial, target or operator duty	Not implemented	Total
DR	8	1	1	10
FR-ACC	10	0	1	11
FR-ADM	10	0	0	10
FR-BRW	12	0	0	12
FR-CAT	20	0	0	20
FR-CRT	6	0	0	6
FR-LNK	14	0	1	15
FR-MSG	8	0	0	8
FR-ORD	14	0	2	16
FR-PAY	8	0	1	9
FR-PRC	10	0	0	10
FR-REV	4	0	0	4
FR-STF	11	0	2	13
FR-SYS	8	0	0	8
IR-EX	11	0	0	11
IR-UI	8	1	0	9
NFR-BKP	0	2	0	2
NFR-CMP	2	1	0	3
NFR-MNT	3	0	0	3
NFR-OPS	2	0	0	2

Area	Implemented	Partial, target or operator duty	Not implemented	Total
NFR-PER	3	1	0	4
NFR-REL	4	1	0	5
NFR-SEC	8	0	2	10
NFR-USE	5	1	0	6
All	**185**	**8**	**10**	**203**

Appendix B: Gaps and future work

Items below are **not implemented** or only **partial**. They are the honest edge of the system today, with a suggested priority.

#	Gap	Related	Suggested priority
1	Two-step sign-in for staff and the super admin	FR-STF-12	High before staff grow in number
2	Phone or email verification at sign-up	FR-ACC-10	High (stops fake accounts and bad contact details)
3	Automatic verification of the shop's price at purchase time for link orders	FR-LNK-11	High (protects margin)
4	Refund started from the admin through the gateway	FR-PAY-09	Medium
5	Customer self-service cancel before buying	FR-ORD-14	Medium
6	Receipts or invoices as downloadable documents	FR-ORD-15	Medium
7	Email invitation for new staff	FR-STF-13	Low
8	Content-Security-Policy on the main pages	NFR-SEC-09	Medium
9	Shared rate-limit store, database server and worker for more than one instance	NFR-SEC-10, DC-1	Only if growth demands it
10	Data export and retention tooling for privacy requests	DR-09, DR-10	Medium
11	Formal accessibility audit (WCAG 2.1 AA)	IR-UI-09, NFR-USE-05	Medium
12	Load testing and measured performance targets	NFR-PER-01	Medium
13	Testing on real iPhone, Android and other browsers	NFR-CMP-01	High, do before launch
14	Volumetric weight for items	FR-PRC-04	Low
15	Multiple languages (for example Twi)	NFR-USE-06	Low
16	Returns and disputes handling	– (out of scope today)	As the business needs
17	Inventory or stock counts for items not from a live source	–	Low (the business holds no stock)

Appendix C: Glossary

Term	Meaning
Catalogue source	A place the importer is allowed to read a shop's products from.
Chargeable weight	The larger of actual and volumetric weight, used to price shipping.
Effective rate	Exchange rate plus the admin's markup; the rate customers pay.
Gateway	A payment provider: Stripe, Paystack or Flutterwave.
Held item	An imported item waiting for staff review.
Link order	An order for an item the customer found elsewhere and sent as a link.
Outbox	The queue of messages awaiting delivery, with retries.
Quote	The UK price and weight of one item, with a private pay link and expiry.
Rate card	A shipping price list by weight.
Right	One thing a staff account may do.
Role	A preset set of rights.
Service charge	The operator's fee for buying on the customer's behalf.
Staff account	An admin sign-in created by the super admin with limited rights.
Super admin	The developer's sign-in using `ADMIN_PASSWORD` ; unlimited and the only one managing staff.
SRS	Software Requirements Specification (this document).
WAL	SQLite's write-ahead log mode.

Appendix D: Revision history

Version	Date	Change
1.0	7 October 2026	First complete, as-built SRS written from the finished system, with every requirement given a status and a means of verification.
1.1	9 October 2026	Added FR-LNK-12 to -15 (ordering from Amazon UK by link, hand-off search, bookmarklet, phone Share menu, requests described in words).

To keep this document true, update the status of a requirement in the same change that implements it, and add new requirements with the next free number in their area.